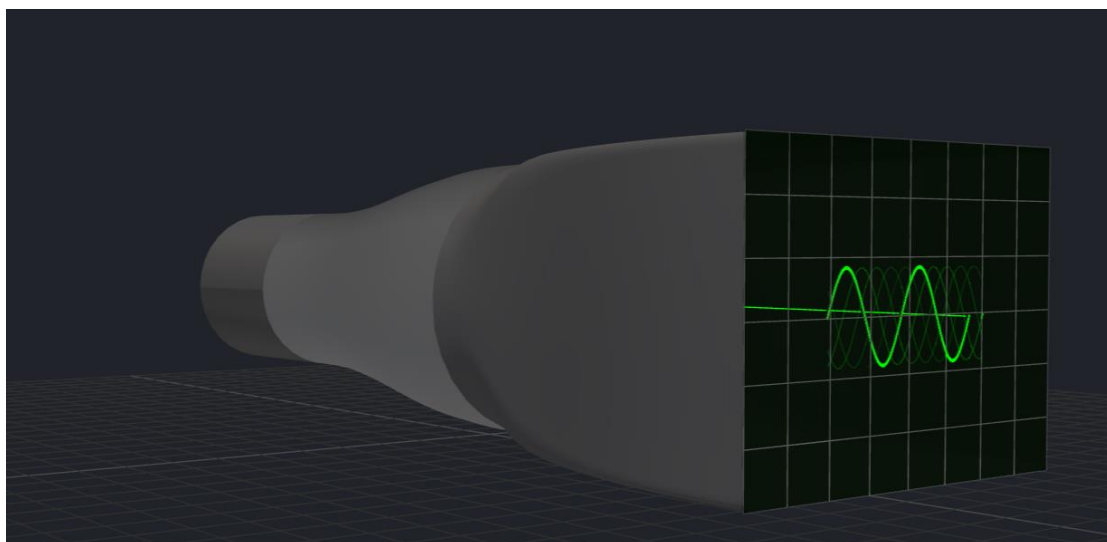


2025 年全国大学生物理实验竞赛（创新赛）

基于 WebGL 的智能示波器交互式虚拟 仿真系统



二〇二五年九月

目录

第一章 作品背景	4
1.1 选题意义.....	4
1.2 目标定位.....	5
第二章 实验原理	5
2.1 示波器图像.....	5
2.1.1 示波器图像生成原理	5
2.1.2 普通正弦图像的获得	5
2.2 简谐振动的合成.....	6
2.2.1 同向同频的简谐振动合成 ^[2]	6
2.2.2 同方向不同频率的简谐振动合成 ^[3]	7
2.2.3 “拍”现象的分析.....	8
2.2.4 相互垂直的简谐振动合成 ^[4]	8
2.2.5 李萨如图 ^[5]	8
2.3 部分算法简介.....	9
2.3.1 超椭圆方程:	9
2.3.2 插值与平滑算法.....	9
第三章 系统设计与实现技术.....	10
3.1 开发流程.....	10
3.1.1 系统架构分析 ^[1]	10
3.1.2 各层处理技术 ^[2]	11
3.2 系统功能模块设计 ^[3]	12
3.2.1 3D 物理仿真模块	13
3.2.2 2D 波形显示模块	13
3.2.3 参数控制模块	14
3.2.4 用户交互模块	14
3.2.5 校准管理模块	15
3.2.6 系统配置模块	15
第四章 实验流程	16
4.1 示波器操作流程.....	16
4.2 示波器功能实现.....	16
4.2.1 示波器基本介绍.....	16
4.2.2 示波器的 2D 使用	17
4.2.3 示波器的 3D 使用	27
4.3 程序使用说明.....	29
4.2.1 软件打开方式(建议):	29
4.2.2 线上打开方式:	30
第五章 项目创新	30

5.1 便携式虚拟实验平台架构创新	30
5.2 智能触发检测算法	30
5.3 多模态显示冲突解决机制	31
5.4 高精度参数调节系统	31
5.5 李萨如图形分析系统	31
第六章 总结与展望	31
6.1 项目收获与成果	31
6.1.1 技术能力的全面提升	31
6.1.2 工程化开发经验积累	31
6.1.3 跨学科知识融合能力	32
6.2 技术创新与突破	32
6.2.1 双页面协同架构创新	32
6.2.2 高精度波形渲染技术	32
6.2.3 智能化校准与触发系统	32
6.3 未来发展展望	32
6.3.1 技术架构升级规划	32
6.3.2 功能扩展与优化	32
6.3.3 教育应用深化	33
第七章 电脑配置与团队分工	33
7.1 电脑配置	33
7.2 团队分工	33
附录：部分程序代码：	33
lissajousDrawer.js 李萨如图绘制：	33
GLWaveformRenderer.js3D 图形库高性能渲染/**	42
CRTShell.jsCRT 外壳页面	44
Screen.js 荧光屏效果和波形显示	58

第一章 作品背景

1.1 选题意义

上世纪 90 年代，德国物理学家布劳恩（Karl Ferdinand Braun）为了捕捉闪电般转瞬即逝的电磁振荡，把电子枪、荧光粉和一对偏转板塞进真空玻璃泡——世界上第一台示波器就此诞生。它像一台“电子显微镜”，让肉眼看不见的电信号第一次留下可见的轨迹。此后 120 余年，从雷达脉冲到心脏起搏、从粒子加速器到 5G 基带，人类每一次高速电现象的破译，几乎都以示波器屏幕上的那道绿色荧光为起点。

然而，当这台“时代之眼”走进大学实验室，却瞬间化身“三高一低”的庞然巨物^[1]：高质量——12kg 的金属机身，搬一次等于做一次硬拉；高价码——中端机型 2 万元起步，经费烧得比信号还快；高门槛——旋钮层级深似海，新手 10 分钟就能把波形“调丢”；低复用——30 人一班，轮候 1 小时，真正上手却只有 3 分钟。



图 1-1 示波器

在 2009 年初，非营利技术联盟 Khronos Group 启动了 WebGL 的工作组。到如今，WebGL 技术已经能把 GPU 算力深度融入浏览器中，在代码编写时我们用两句简短的代码：

```
vec2 clipSpace = (a_position/u_resolution) × 2.0 - 1.0;
gl_Position = vec4(clipSpace × vec2(1, -1), 0, 1);
```

即可在屏幕上“渲染”出可无限缩放、实时交互的波形——正弦、方波、三角、李萨如图，乃至任意自定义信号，皆可在极短时间内完成一次刷新。

本研究的核心目标是为高等院校的大学生打造一个便携式、高效交互的虚拟实验环境。在这个平台上，学生能够通过调整各种参数，直观观察不同波形的显示特性，以及李萨如图形在不同频率比和相位差下的变化规律。

更为重要的是，学生可以通过自检校准、标准测量等实验步骤，系统地掌握示波器的操作技能，深入理解示波器的工作原理，为后续实际操作真实仪器奠定坚实基础。这种虚拟实验方式不仅突破了传统实验室的时空限制，也为教学资源紧张的学校提供了一种经济高效的教學解决方案。

针对上述情况，本竞赛组开发了一款“理论可视、操作可练、结果可验”的综合性示波器仿真系统，通过高度仿真的交互体验，帮助学生在虚拟环境中掌握示波器的核心操作技能，深入理解波形测量原理，特别是李萨如图形形成的物理机制，为后续的实际仪器操作

打下坚实基础。该仿真系统开发所用软件为 VS Code，使用的编程语言为 JavaScript，最终将代码上传到 GitHub，打包成为一个独立程序，可在使用者电脑上运行。

1.2 目标定位

本项目是一款基于 Three.js 与 WebGL 技术构建的"零部署、高保真、3D 可视"示波器仿真教学平台。通过 Vue.js 响应式框架和现代 Web 技术，在浏览器中完整还原真实 CRT 示波器的操作体验，集成双通道波形显示、精密触发系统、自检校准流程等专业功能。

核心创新在于 3D 内部原理可视化：真实模拟电子枪发射、偏转板控制、荧光屏发光等物理过程，配合分解动画和交互标签系统，让学生直观理解示波器工作机制。支持六种波形类型生成、李萨如图形显示、实时参数调节，涵盖从基础校准到高级测量的完整实验流程。

技术架构采用模块化设计：Electron 桌面应用 + Web 版双模式部署，Webpack 5 构建优化，确保跨平台兼容性。内置演示动画、导览系统、参数预设等智能化教学工具，实现"调参-测量-自检"的快速学习闭环。

解决传统教学痛点：设备昂贵难普及、操作复杂上手难、原理抽象理解难。通过"一键运行、零成本部署、沉浸式体验"的设计理念，让学生在 10 分钟内掌握示波器核心操作，为电子工程教育提供高效、直观的数字化解决方案。

第二章 实验原理

2.1 示波器图像

2.1.1 示波器图像生成原理

示波器通过电子束在荧光屏上的动态扫描实现电信号波形的可视化^[1]。其核心部件为示波管，由电子枪、偏转系统和荧光屏组成。电子枪通过灯丝加热阴极发射电子，经栅极控制电子束密度（辉度调节），并由聚焦电极（第一阳极 A1 和第二阳极 A2）将电子束聚焦成细束。垂直偏转板（Y 轴）和水平偏转板（X 轴）分别施加信号电压和扫描电压，控制电子束在荧光屏上的位置。

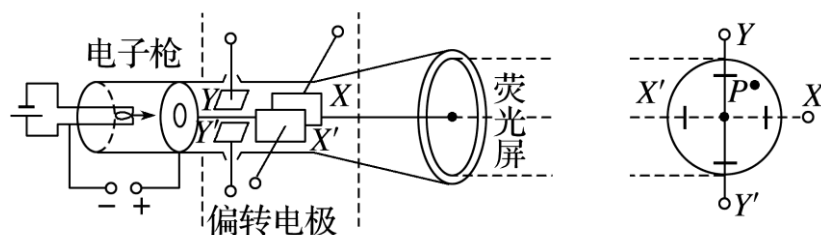


图 2-1 示波管结构示意图

2.1.2 普通正弦图像的获得

示波器时变电压可视化：电子束在垂直方向受信号驱动，在水平方向匀速扫描，即可在荧光屏上“描绘”出电压随时间变化的轨迹。

电子束在垂直方向随信号电压上下移动正弦信号：

$$U_y(t) = U_{y0} \sin(\omega t)$$

电子束在水平方向以恒定速度扫描：

$$U_x(t) = k \cdot t$$

扫描电压的周期 T_{scan} 需与被测信号周期 T_{sig} 满足：

$$T_{scan} = n T_{sig} \quad (n \in \mathbb{Z}^+)$$

上述三个公式整合在一起，可以得到示波器屏幕上正弦图像运动的参数方程表示：

$$\begin{cases} U_x(t) = k \cdot t \\ U_y(t) = U_{y0} \sin\left(\frac{2\pi}{T_{sig}}t\right) \end{cases} \quad \text{且满足} \quad T_{scan} = n \cdot T_{sig}$$

2.2 简谐振动的合成

在物理学中，当两个或多个同方向且同频率的简谐振动作用于同一物体时，会发生简谐振动的合成现象。以下是对不同情况下简谐振动合成的详细阐述：

2.2.1 同向同频的简谐振动合成^[2]

假设存在两个同方向的简谐振动，其振动方程分别为：

$$x_1(t) = A_1 \cos(\omega t + \phi_1)$$

$$x_2(t) = A_2 \cos(\omega t + \phi_2)$$

其中， A_1 、 A_2 分别为两者的振幅， ω 为共同的角频率， ϕ_1 、 ϕ_2 为初相位。合成后的振动位移可表示为：

$$x = x_1 + x_2 = A_1 \cos(\omega t + \phi_1) + A_2 \cos(\omega t + \phi_2)$$

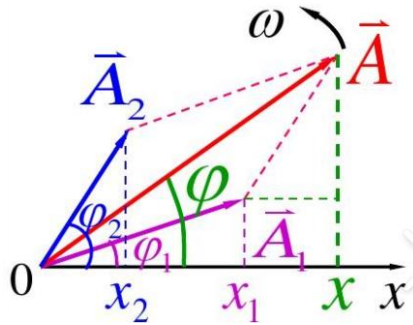


图 2-2 同向同频的简谐振动合成

通过三角函数合成公式，可将其转化为：

$$x(t) = A \cos(\omega t + \phi)$$

两个同方向同频率的简谐振动合成后仍为简谐振动。合成后的振幅：

$$A = \sqrt{A_1^2 + A_2^2 + 2A_1A_2 \cos(\phi_2 - \phi_1)}$$

初相位：

$$\tan \varphi = \frac{A_1 \sin \varphi_1 + A_2 \sin \varphi_2}{A_1 \cos \varphi_1 + A_2 \cos \varphi_2}$$

当两分振动同相位（即 $\varphi_1=\varphi_2$ ）时，振幅最大，为两分振幅之和。反相位（即 $\varphi_1=\varphi_2+\pi$ ）时，振幅最小，为两分振幅之差的绝对值，若两分振幅相等，则合成后的振动振幅为零，振动停止。

2.2.2 同方向不同频率的简谐振动合成^[3]

对于同方向但不同频率的两个简谐振动合成，情况更为复杂。设其振动方程分别为：

$$x_1 = A_1 \cos \omega_1 t$$

$$x_2 = A_2 \cos \omega_2 t$$

合振动：

$$x = x_1 + x_2$$

$$= 2A \cos \left(\frac{\omega_1 - \omega_2}{2} t \right) \cos \left(\frac{\omega_1 + \omega_2}{2} t + \phi \right)$$

角频率：

$$\frac{\omega_1 + \omega_2}{2}$$

振幅：

$$\left| 2A \cos \left(\frac{\omega_1 - \omega_2}{2} t \right) \right|$$

所形成的图像为：

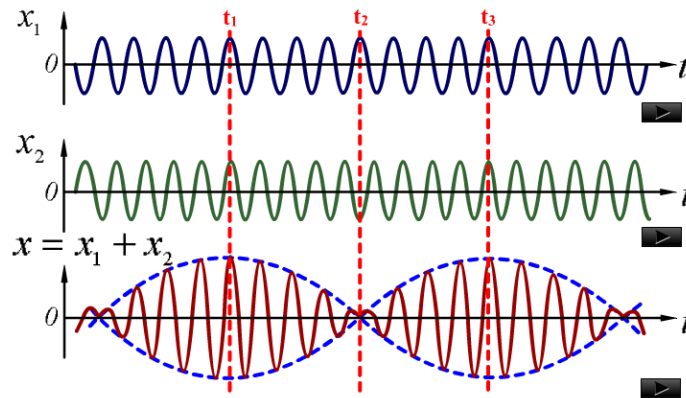


图 2-3 同向不同频的简谐振动合成

其中， $\omega_1 \neq \omega_2$ 。合成后的振动不再是简谐振动，而是一个周期性的复杂振动。由于两分振动的频率不同，其相位差会随时间变化，这使得合成后的振动振幅和相位也随之发生周期性变化，形成一种称为“拍”的现象。在一定时间范围内，合成振动的振幅时而增强，时而减弱。这种现象在声学领域有广泛应用，例如通过听音辨频时，可通过拍现象来判断和调整频率，实现精准的音准控制。

2.2.3 “拍”现象的分析

当 ω_1 与 ω_2 相近时，合振幅 $A_{(t)}$ 的变化频率为：

$$v = |v_2 - v_1| = \frac{|\omega_2 - \omega_1|}{2\pi}$$

其周期为：

$$T = \frac{1}{v} = \frac{2\pi}{|\omega_2 - \omega_1|}$$

此时，合振动的振幅时而增强，时而减弱。这种现象在声学领域有广泛应用，例如通过听音辨频时，可通过拍现象来判断和调整频率，实现精准的音准控制。

2.2.4 相互垂直的简谐振动合成^[4]

当两个简谐振动的振动方向相互垂直时，合成后的运动轨迹将具有独特的特征。设这两个相互垂直的简谐振动方程分别为：

$$x(t) = A_x \cos(\omega t + \phi_x)$$

$$y(t) = A_y \cos(\omega t + \phi_y)$$

质点运动轨迹：

$$\frac{x^2}{A_1^2} + \frac{y^2}{A_2^2} - \frac{2xy}{A_1 A_2} \cos(\varphi_2 - \varphi_1) = \sin^2(\varphi_2 - \varphi_1)$$

两相互垂直同频率不同相位差简谐运动合成图：

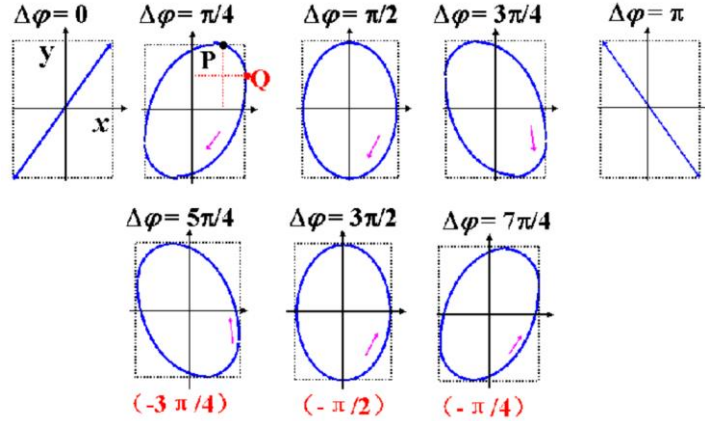


图 2-4 相互垂直的简谐振动合成

合成后的运动轨迹由这两个振动的频率比和相位差共同决定。

当频率比为：

$$\omega_x : \omega_y = m : n, \quad m, n \in \mathbb{Z}$$

时，轨迹为稳定的闭合曲线，称为李萨如图形（Lissajous Figures）

2.2.5 李萨如图^[5]

当频率比为：

$$\omega_x : \omega_y = m : n, \quad m, n \in \mathbb{Z}$$

轨迹为稳定的闭合曲线，称为李萨如图形。

图形形状由频率比和相位差

$$\Delta\phi = \phi_y - \phi_x$$

共同决定。不同的频率比和相位差会产生形态各异李萨如图形，这些图形在测量频率、相位差等方面有着重要的应用。

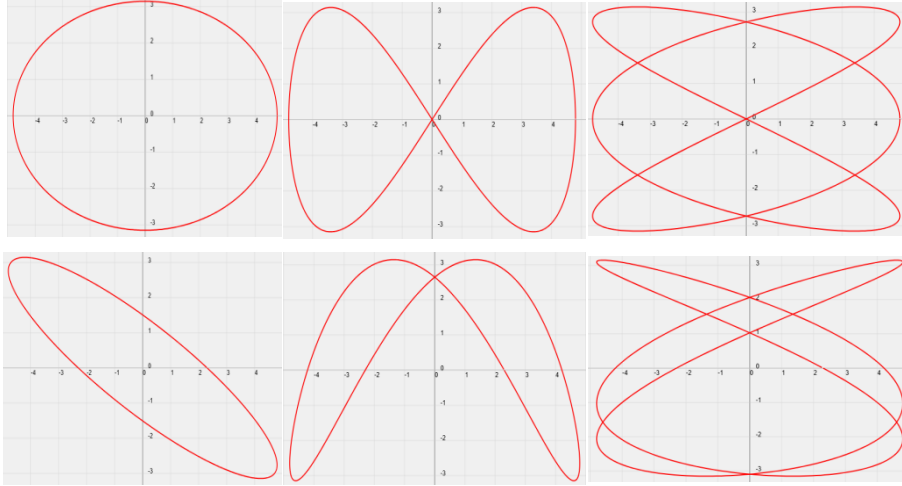


图 2-5 不同频率比、初相差下的李萨如图

2.3 部分算法简介

2.3.1 超椭圆方程：

$$|x/a|^m + |y/b|^m = 1$$

参数定义：

a、b 分别为 x 轴、y 轴方向的“半长轴”，控制超椭圆的整体尺寸；m 为“形状指数”，决定图形从“圆形”到“方形”的渐变趋势。

渐变规律：

- 当 $m=2$ 时，方程退化为椭圆（若 $a=b$ ，则为圆形）；
- 当 $m>2$ 时，图形边缘逐渐“趋直”， m 越大，越接近矩形（如 $m=5$ 时近似方形， $m \rightarrow \infty$ 时为严格矩形）；
- 当 $1<m<2$ 时，图形边缘呈“圆角菱形”， $m=1$ 时为菱形（ $a=b$ 时为正方形）。

2.3.2 插值与平滑算法

1. G1 连续性插值（三次平滑插值）

G1 连续性指“位置连续 + 一阶导数连续”，适用于对“平滑度要求中等”的轨迹（如机械臂路径），采用公式：

$$S(s) = 3s^2 - 2s^3 \quad (s \in [0, 1])$$

特性：

位置连续：s=0 时 $S(0)=0$ ，s=1 时 $S(1)=1$ ，可通过平移缩放适配任意两点间插值；

一阶导数连续： $S'(s)=6s(1-s)$ ，s=0 时 $S'(0)=0$ ，s=1 时 $S'(1)=0$ ，相邻插值段在连接点的导数一致。

2. G2 连续性插值（五次平滑插值）

G2 连续性指“位置连续 + 一阶导数连续 + 二阶导数连续”（曲率连续），适用于对“平滑度要求高”的场景（如汽车轨迹、流体模拟），采用公式：

$$S(s) = 10s^3 - 15s^4 + 6s^5 \quad (s \in [0, 1])$$

特性：

二阶导数连续： $S''(s)=60s(1-s)(1-2s)$ ， $s=0$ 时 $S''(0)=0$ ， $s=1$ 时 $S''(1)=0$ ，曲率无突变；

平滑度：相比三次插值，五次插值曲线更“柔和”，无局部凸起。

3. 线性插值（Lerp）

线性插值是“无平滑过渡”的基础插值方式，适用于参数线性变化场景（如颜色过渡、位置渐变），采用 Three.js 内置函数 `THREE.MathUtils.lerp(start, end, t)`，原理为：

$$L(t) = start + (end - start) \cdot t \quad (t \in [0, 1])$$

特性：

$t=0$ 时返回 `start`， $t=1$ 时返回 `end`， $t=0.5$ 时返回两点中点。

4. 轨迹平滑（加权平均算法）

针对电子束、传感器等离散轨迹的“噪声消除”，采用加权平均平滑，原理为：

设原始轨迹点为 $P_0, P_1, \dots, P_n$ ，平滑后第 i 个点 P'_i 为：

$$P'_i = \sum_{k=-k_0}^{k_0} w_k \cdot P_{i+k}$$

约束：权重满足

$$\sum_{k=-k_0}^{k_0} w_k = 1$$

常用高斯权重（如

$$w = [0.1, 0.2, 0.4, 0.2, 0.1]$$

），权重越集中于中心，平滑幅度越小。

第三章 系统设计与实现技术

3.1 开发流程

3.1.1 系统架构分析^[1]

示波器仿真系统主要由 3D 物理仿真模块、2D 波形分析模块、用户交互控制、配置管理、材质渲染、状态管理、组件管理、导览引导和模块切换器组成。当用户访问系统时，通过传输层 **HTTP/HTTPS** 协议进行资源请求和响应，使用**现代浏览器安全机制**对跨域访问进行控制，严格保障系统运行安全。应用层为不同仿真功能提供相应的**专业化服务模**

块，数据层使用 JSON 配置文件存放系统参数和物理仿真数据，将实时状态数据和用户交互数据放入 LocalStorage 本地存储，显著提高系统的响应速度和用户体验。

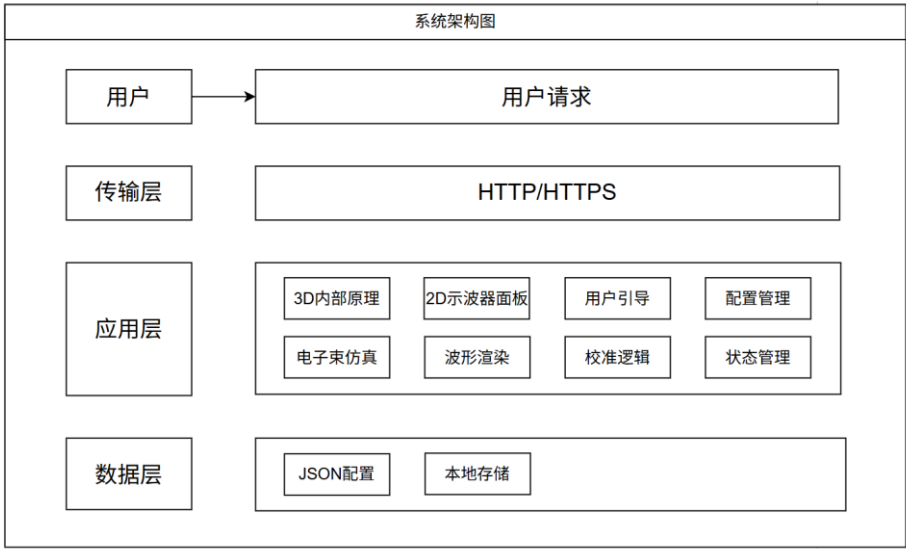


图 3-1 系统详细架构图

3.1.2 各层处理技术^[2]

本系统采用 C/S 架构（也可部署为 B/S 架构）和模块化设计模式，基于 Webpack 构建框架和 Vue.js 框架，实现了前端模块化开发。前端采用的技术有 Vue.js 2.7.16 框架、Three.js 0.177.0 图形库、dat.GUI 0.7.9 控制面板、TWEEN.js 25.0.0 动画库，渲染引擎采用的技术有 WebGL 硬件加速、Canvas 2D 绘图、ES6+模块系统、热重载技术、数学算法库、波形插值算法和电子束轨迹计算算法等。

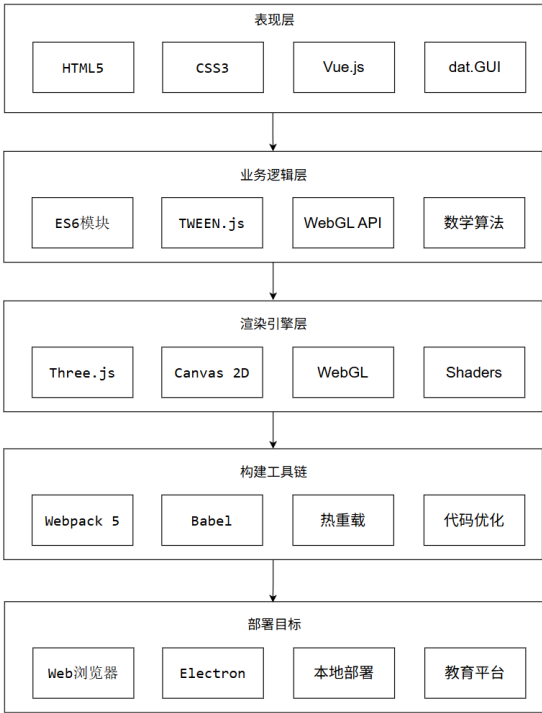


图 3-2 各层处理技术详细表

本示波器仿真系统采用现代化五层架构设计：

表现层基于 HTML5 语义化结构和 CSS3 响应式设计构建美观界面，Vue.js 提供数据驱动的交互逻辑，dat.GUI 实现专业参数调节功能；

业务逻辑层运用 ES6 模块化架构保证代码可维护性，TWEEN.js 提供平滑动画效果，结合 WebGL API 底层调用和物理仿真数学算法实现精确的电子束轨迹计算；

渲染引擎层以 Three.js 构建逼真 3D 示波器模型，Canvas 2D 实现高性能实时波形绘制，WebGL 提供 GPU 加速渲染，自定义着色器营造电子束发光等视觉效果；

构建工具链采用 Webpack 5 进行模块化打包和代码优化，Babel 确保 ES6+语法的浏览器兼容性，支持热重载提升开发效率；

部署目标支持 Web 浏览器直接在线使用、Electron 跨平台桌面应用覆盖 Windows/Mac/Linux 系统、本地内网部署以及教育平台集成，形成了技术先进、教学实用的完整仿真教学解决方案。

3.2 系统功能模块设计^[3]

本文研究的示波器仿真系统主要包括 **6 个功能模块**，分别是：**3D 物理仿真模块**、**2D 波形显示模块**、**参数控制模块**、**校准管理模块**、**用户交互模块**和**系统配置模块**。基于本人在电子信息类专业教学中遇到的一些问题，因传统示波器教学设备昂贵、数量有限，且学生难以直观理解示波器内部工作原理，理论学习与实践操作脱节，严重影响了教学效果和学生的学习积极性。该系统将着力解决这些问题，通过**创新的双重体验模式**和**可视化教学方法**，从根本上提升示波器教学的质量和效率。

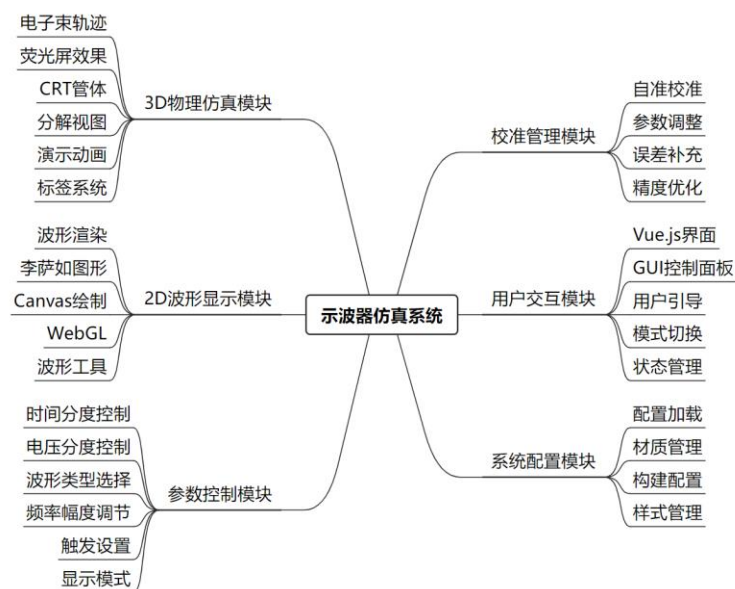


图 3-3 详细模块记录图

3.2.1 3D 物理仿真模块

该模块是系统的核心，基于 Three.js 构建了精确的阴极射线管 3D 模型，包含电子枪、荧光屏效果、CRT 管体、分解视图、波形动画和标签系统等关键组件。通过 SuperellipseTransition 实现超椭圆形状的平滑过渡，CylinderConnection 处理旋转曲面连接，ExplodedView 提供爆炸分解视图功能。系统运用复杂的数学算法模拟电子束在电磁场中的偏转轨迹，实现了物理原理的精确可视化，为用户提供沉浸式的学习体验。

```
// src/main.js - 3D物理仿真核心代码
import { CRTShell } from './components/CRTShell';
import { ElectronBeam } from './components/ElectronBeam';
import { Screen } from './components/Screen';
import { ExplodedView } from './components/ExplodedView';
import { DemoAnimation } from './components/DemoAnimation';

// 创建3D场景和核心组件
function init() {
  scene = new THREE.Scene();
  camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 1000);
  renderer = new THREE.WebGLRenderer({ antialias: true });

  // 初始化核心3D组件
  crtShell = new CRTShell();
  electronBeam = new ElectronBeam(scene);
  screenController = new Screen(scene);
  explodedView = new ExplodedView(scene, components);
  demoAnimation = new DemoAnimation(scene, components, controllers);
}
```

3.2.2 2D 波形显示模块

此模块专门负责波形的生成、渲染和显示功能，采用 Canvas 2D 技术实现高性能的实时波形绘制。WaveformGenerator 类能够生成正弦波、方波、三角波等多种标准信号，支持频率、幅度、相位等参数的实时调节。WebGL 技术确保波形显示的流畅性，波形工具提供缩放、测量等交互功能，时间分度控制实现精确的时间轴管理，为电子技术教学提供专业级的波形分析环境。

```
// src/external.js - 2D波形显示核心代码
import { WaveDrawer } from '../scripts/waveDrawer.js';
import LissajousDrawer from '../scripts/lissajousDrawer.js';

// Canvas波形绘制
function drawWaveform() {
  const canvas = this.$refs.oscilloscope;
  const ctx = canvas.getContext('2d');

  // 清除画布并绘制网格
  this.clearCanvas();
  this.drawGrid();

  // 绘制波形
  if (this.displayMode === 'vertical') {
    LissajousDrawer.drawLissajous(ctx, this.freqX, this.freqY, this.phaseDiff);
  } else {
    WaveDrawer.drawWave(ctx, this.signalType, this.timeDiv, this.voltsDiv);
  }
}
```

3.2.3 参数控制模块

该模块构建了完整的用户交互控制系统，采用 dat.GUI 实现专业的参数调节面板，支持垂直和水平偏转电压的实时控制。时间分度控制允许用户调节时间轴精度，电压参数调整可实时观察电子束轨迹变化，波形类型选择提供多种信号源切换。触发设置功能确保波形稳定显示，显示模式切换支持不同的观察视角，为用户提供全面的实验操作体验。

```
// src/controllers/GuiController.js - 参数控制核心代码
export class GuiController {
  createGUI() {
    this.gui = new dat.GUI();

    // 偏转电压控制
    const voltageFolder = this.gui.addFolder('偏转电压');
    voltageFolder.add(this.params, 'verticalVoltage', -5, 5, 0.1)
      .name('垂直偏转(V)')
      .onChange((value) => this.updateDeflection());

    voltageFolder.add(this.params, 'horizontalVoltage', -5, 5, 0.1)
      .name('水平偏转(V)')
      .onChange((value) => this.updateDeflection());

    // 波形参数控制
    const waveFolder = this.gui.addFolder('波形参数');
    waveFolder.add(this.params, 'frequency', 0.1, 10, 0.1)
      .name('频率(Hz)')
      .onChange((value) => this.updateWaveform());
  }
}
```

3.2.4 用户交互模块

此模块专注于提升用户体验，基于 Vue.js 框架构建响应式界面，实现了自适应的参数调整功能。误差补偿机制确保操作的准确性，精度优化提升交互响应速度。GUI 控制面板集成了所有操作功能，用户指导系统提供实时帮助信息，模式切换功能支持不同学习场景，状态管理确保操作的一致性，为教学和学习提供友好的交互环境。

```
// src/external.js - Vue用户交互核心代码
function createVueApp() {
  return new Vue({
    el: '#app',
    data: {
      signalType: 'sine',
      inputActive: { 1: false, 2: false },
      displayMode: 'independent',
      timeDiv: 1,
      voltsDiv: { 1: 1, 2: 1 },
      triggerLevel: 0
    },
    methods: {
      // 通道切换
      toggleInput(line) {
        this.inputActive[line] = !this.inputActive[line];
        this.refreshDisplay();
      },

      // 显示模式切换
      setDisplayMode(mode) {
        this.displayMode = mode;
        if (mode === 'vertical') this.currentMode = 'lissajous';
        this.refreshDisplay();
      },

      // 参数调整
      adjustParam(param, delta, channel = null) {
        // 实时参数调整逻辑
        this.refreshDisplay();
      }
    }
  });
}
```

3.2.5 校准管理模块

该模块负责系统的精度控制和校准功能，确保仿真结果的准确性。自准校准功能自动调整系统参数，参数调整机制实现精细化控制，误差补充算法修正计算偏差。精度优化确保物理仿真的准确性，配置加载系统管理各种预设参数，材质管理统一处理 3D 模型的渲染效果，构建管理维护整体架构的稳定性，为专业教学应用提供可靠的技术保障。

```
// scripts/calibrationLogic.js - 校准管理核心代码
export default class CalibrationLogic {
  constructor() {
    this.calibrationParams = {
      peakValues: { 1: 4, 2: 4 },
      frequencies: { 1: 1, 2: 1 },
      calibrationFactor: 0.85
    };
  }

  // 自动校准功能
  performAutoCalibration() {
    const referenceSignal = this.generateReferenceSignal();
    const measurementError = this.calculateMeasurementError();

    // 误差补偿算法
    this.applyErrorCompensation(measurementError);

    // 精度优化
    this.optimizePrecision();
  }

  // 参数精度管理
  validateAndAdjustParameters(params) {
    // 参数验证和自动调整逻辑
    return this.applyPrecisionControl(params);
  }
}
```

3.2.6 系统配置模块

此模块管理整个系统的运行环境和部署配置，基于 Electron 技术实现跨平台桌面应用打包。构建管理使用 Webpack 5 进行模块化打包优化，样式管理统一处理 CSS 样式和主题切换。配置文件系统管理各种参数设置，热重载功能提升开发效率，代码优化确保运行性能，部署脚本支持一键发布到多个平台，为系统的维护和升级提供完整的工程化解决方案。

```
// webpack.config.js - 系统构建配置核心代码
module.exports = {
  entry: {
    internal: './src/main.js', // 3D内部原理页面入口
    external: './src/external.js' // 2D外部面板页面入口
  },
  output: {
    path: path.resolve(__dirname, 'docs'),
    filename: 'js/[name].bundle.js'
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        use: 'babel-loader',
        exclude: /node_modules/
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader']
      }
    ]
  }
};
```

```
// packaging/electron-main.js - Electron主进程配置
const { app, BrowserWindow } = require('electron');

function createWindow() {
  const mainWindow = new BrowserWindow({
    width: 1200,
    height: 800,
    webPreferences: {
      nodeIntegration: true,
      contextIsolation: false
    }
  });
}

// 加载构建后的应用
mainWindow.loadFile('docs/index.html');
}
```

第四章 实验流程

4.1 示波器操作流程

该程序以示波器仿真实验展开，基本操作流程如下：

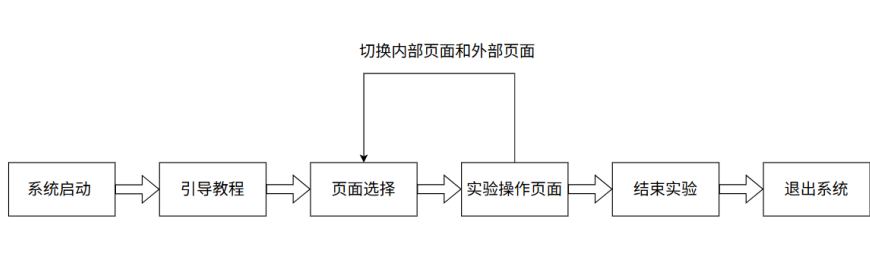


图 4-1 示波器虚拟仿真系统基础操作流程

4.2 示波器功能实现

4.2.1 示波器基本介绍

[1] 引导框介绍：

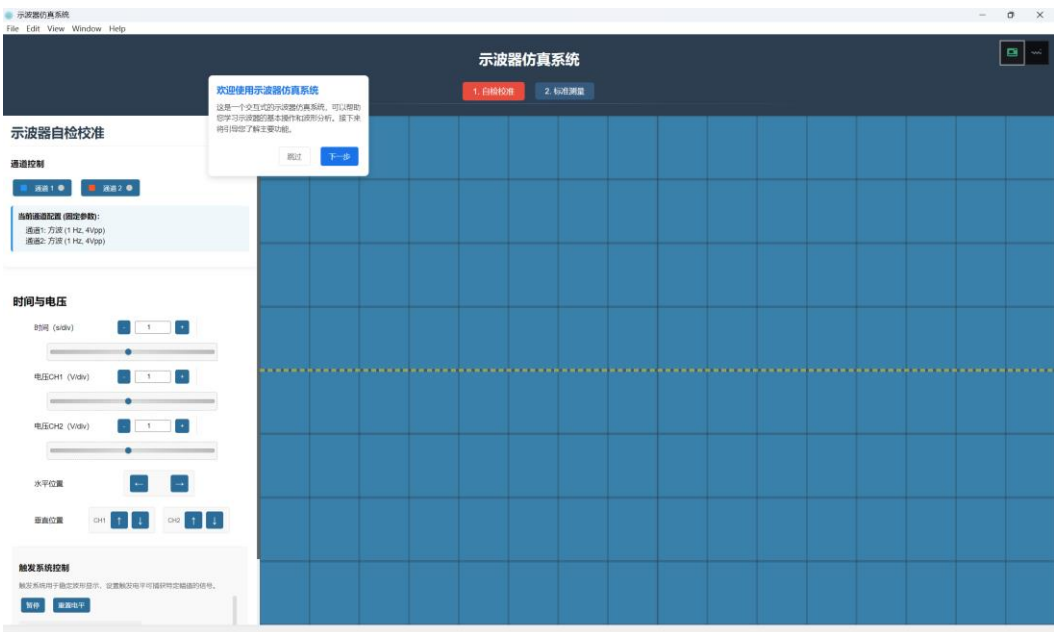


图 4-2 引导行界面

[2] 系统启动：

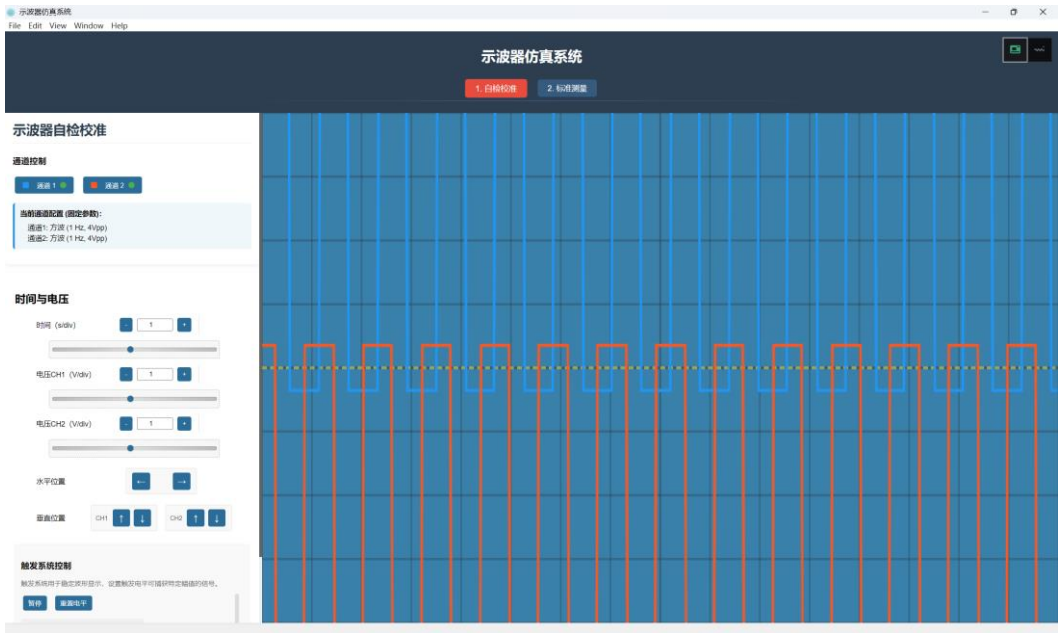


图 4-3 自检初始界面

[3] 界面熟悉：

左侧：控制面板区域，包含参数设置和模式切换

右侧：示波器显示区域，显示波形或李萨如图形

上方：实验步骤切换按钮（自检校准和标准测量）

在右侧顶部：2D/3D 切换

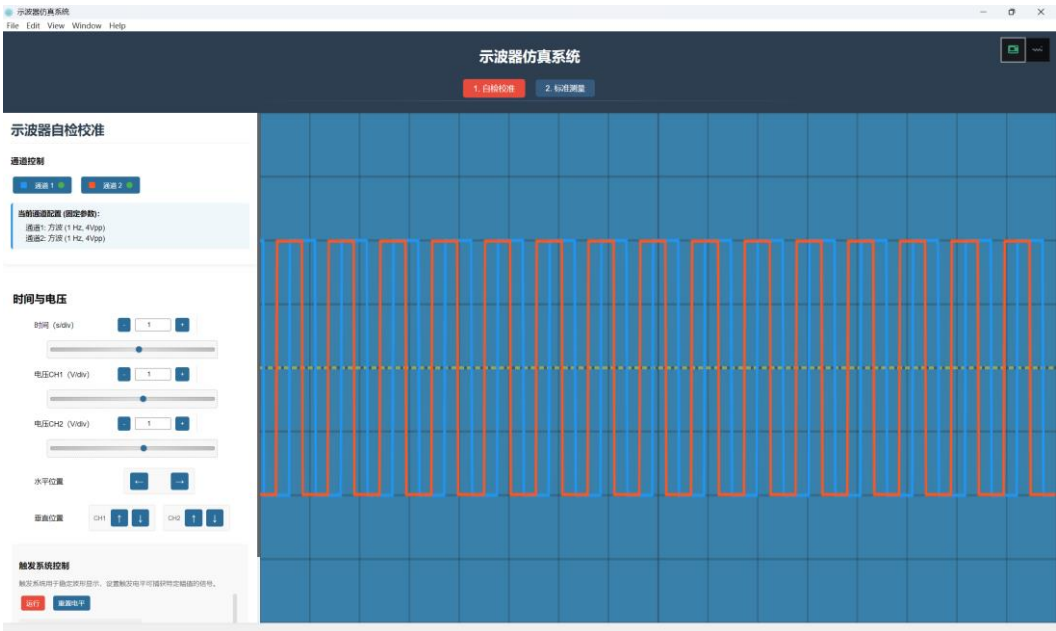


图 4-4 熟悉界面

4.2.2 示波器的 2D 使用

Step0 校准操作：

- [1] 进入自检模式：点击"**1. 自检校准**"按钮进入校准模式，系统默认输出方波信号（频率 1Hz，峰峰值 4Vpp）

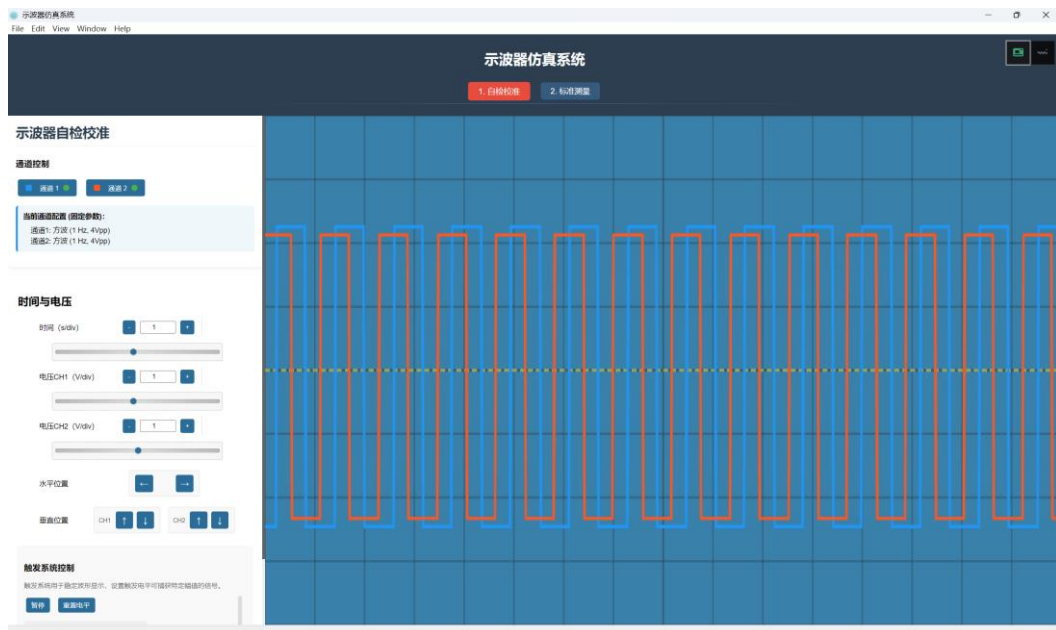


图 4-5 自检前的照片

其中时间“1”，单位是秒 / 格。这是示波器水平时间基准的设置，在实际示波器中，它决定了每个水平格所代表的时间长度。例如，如果时间设置为 1s/div，那么示波器屏幕水平方向上的一格就对应 1 秒的时间跨度。

电压“1”，单位是伏特 / 格。这是示波器垂直电压灵敏度的设置，用于确定每个垂直格所代表的电压值。

“水平位置”和“垂直位置”的滑块可以调整波形在示波器屏幕上的位置，通过滑块的移动来达成示波器校准的功能实现

- [2] 通道激活：

点击"**通道 1**"和"**通道 2**"按钮，激活对应通道

观察屏幕上显示的方波信号



图 4-6 通道激活

- [3] 校准操作：

调整"时间微调"滑块，使波形边缘尽量垂直

调整"通道 1 电压微调"和"通道 2 电压微调"滑块，使波形高度合适

注意观察波形形状变化，直到达到理想的方波形状

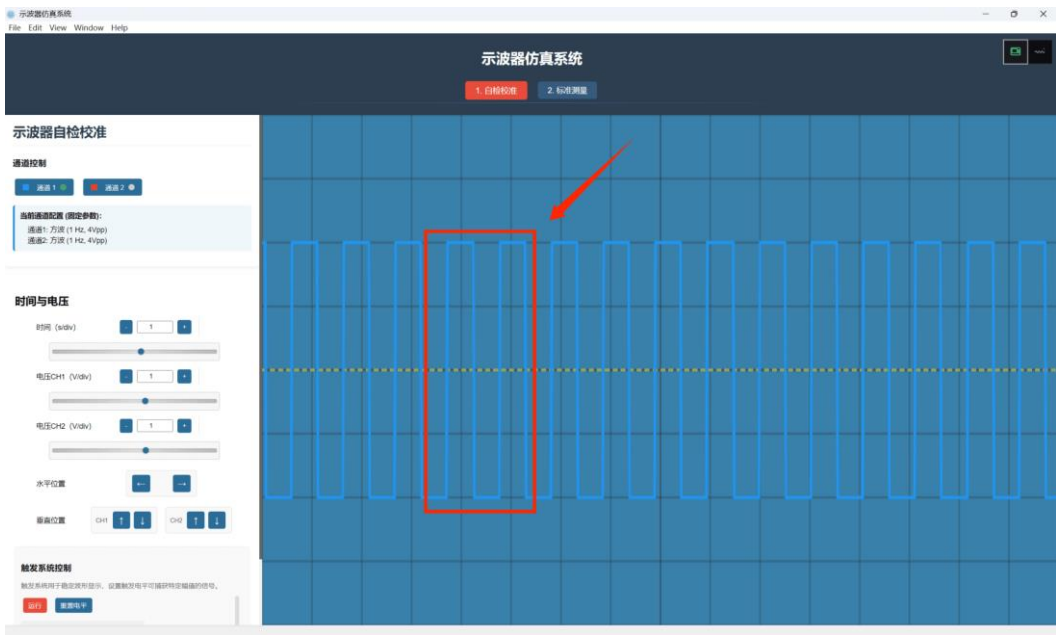


图 4-7 校准中照片

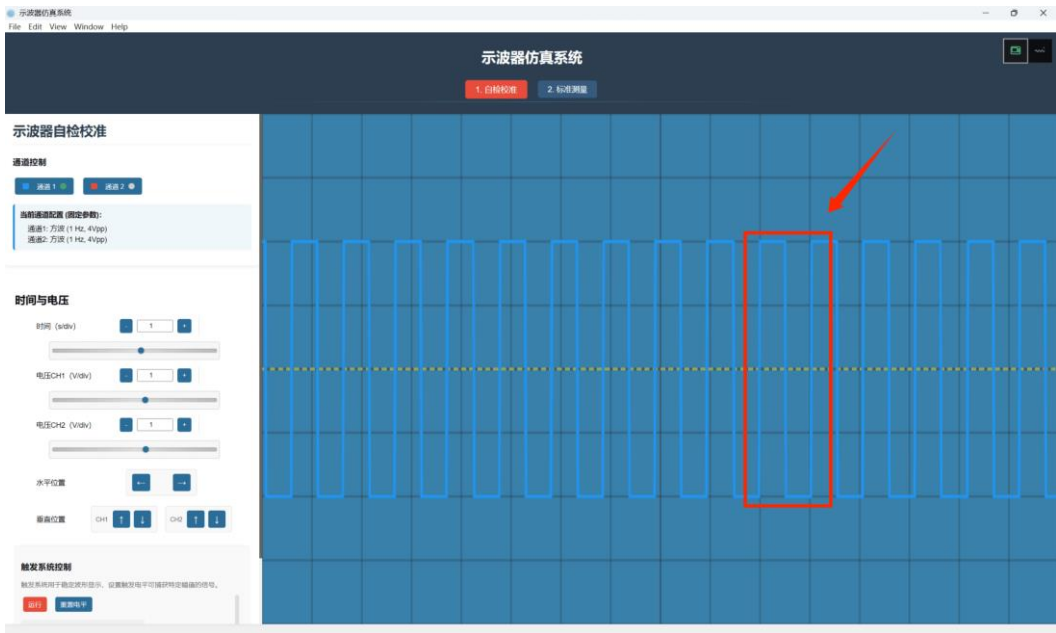


图 4-8 校准成功照片*

在图中可以发现水平方向正好为 1 个空格对应一个周期，垂直方向 4V 对应着四格的方块，则代表机器校准成功，可继续的工作。

Step1 标准测量实验：

[1] 测量模式启动：

点击"2. 标准测量"按钮进入实验模式，确认系统校准参数已正确保存，确保波形呈现稳定清晰的状态。

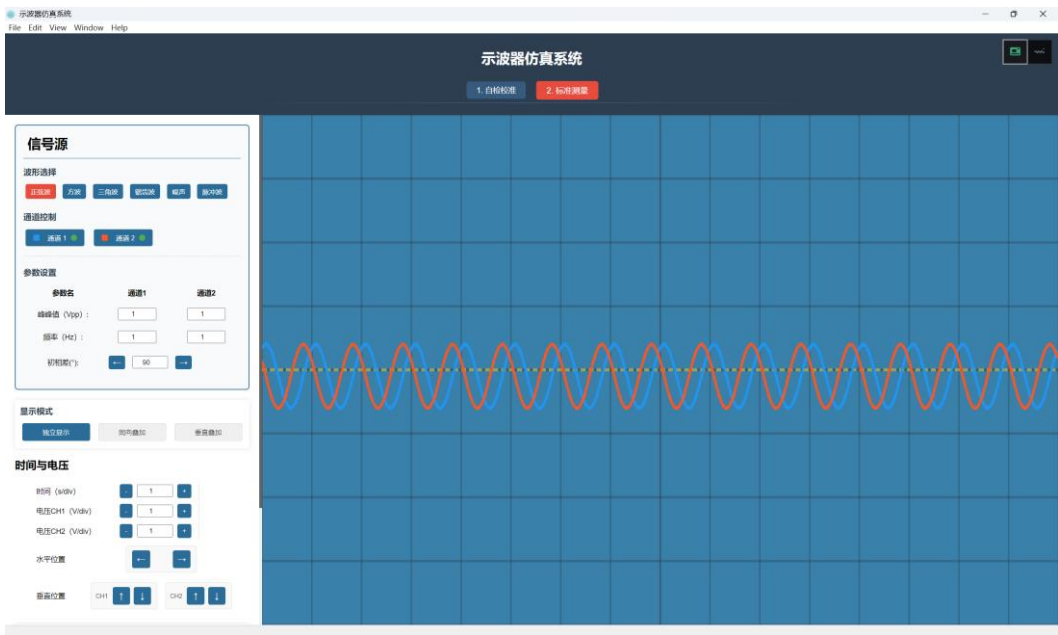


图 4-9 测量初始界面

[2] 波形精细调整：

时间分度调整 (s/div)：合理选择时间轴比例，如频率为 1Hz 时推荐 0.2s/div，此时波形周期占据约 5 格。

电压分度调整 (V/div)：适当调整电压比例，例如 4Vpp 的波形推荐设置 1V/div，使波形垂直方向占 4 格。

波形位置微调：使用箭头按钮进行精确的水平及垂直位置调整。

在标准测量中包含多种波形：

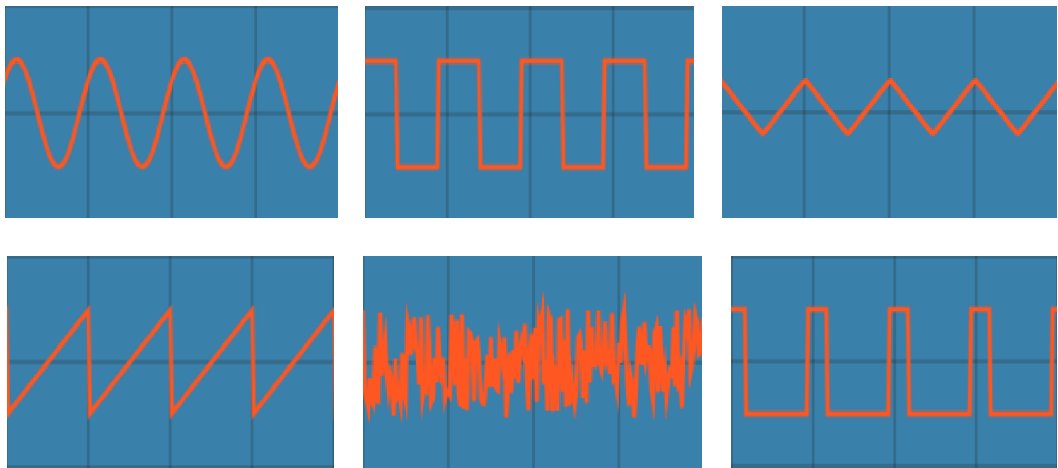


图 4-10 智能示波器下多种波形展示

Step2 双通道测量分析：

同频信号测量：通道 1 和通道 2 频率均设为 1Hz，调整相位差至 90°，确保通道 2 波形比通道 1 滞后 1/4 周期。

不同频信号测量：通道 1 设为 1Hz，通道 2 设为 2Hz，观察并验证频率比 1：2 时的波

形特征。

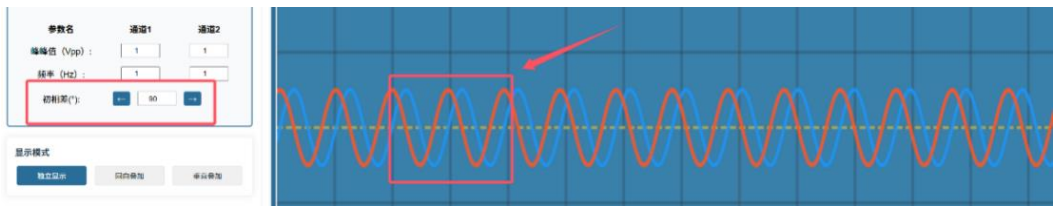


图 4-11 相位差 90 所形成图像*

Step3 触发系统的精确控制：

触发电平设置：初始设置为 0V，通过上下调整触发阈值观察波形变化情况，最优触发位置通常设置在信号上升沿中点。

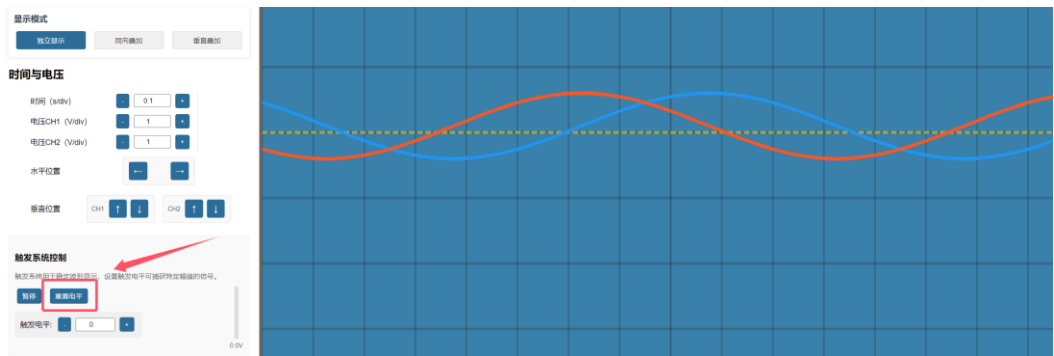


图 4-12 重置电平

波形刷新控制：使用运行/暂停按钮实现波形实时刷新或静态观察，暂停模式下触发电平的调整不会影响当前显示波形。

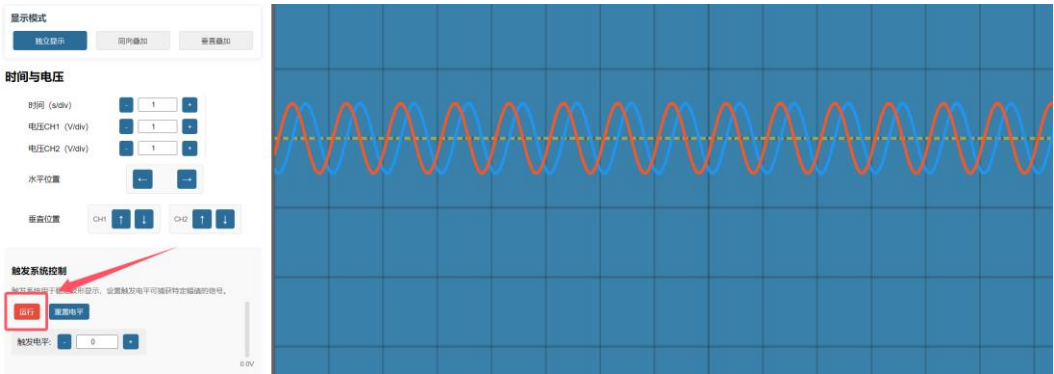


图 4-13 运行

Step4 高级波形特征研究：

复合波形叠加分析：采用同向叠加模式显示并分析两通道波形间的相位差异。

频率响应特性测试：固定通道 1 的频率，逐步提高通道 2 的频率（如 1Hz 逐渐增加至 5Hz），系统性观察屏幕上显示波形周期数的变化，深入理解信号频率响应特征。

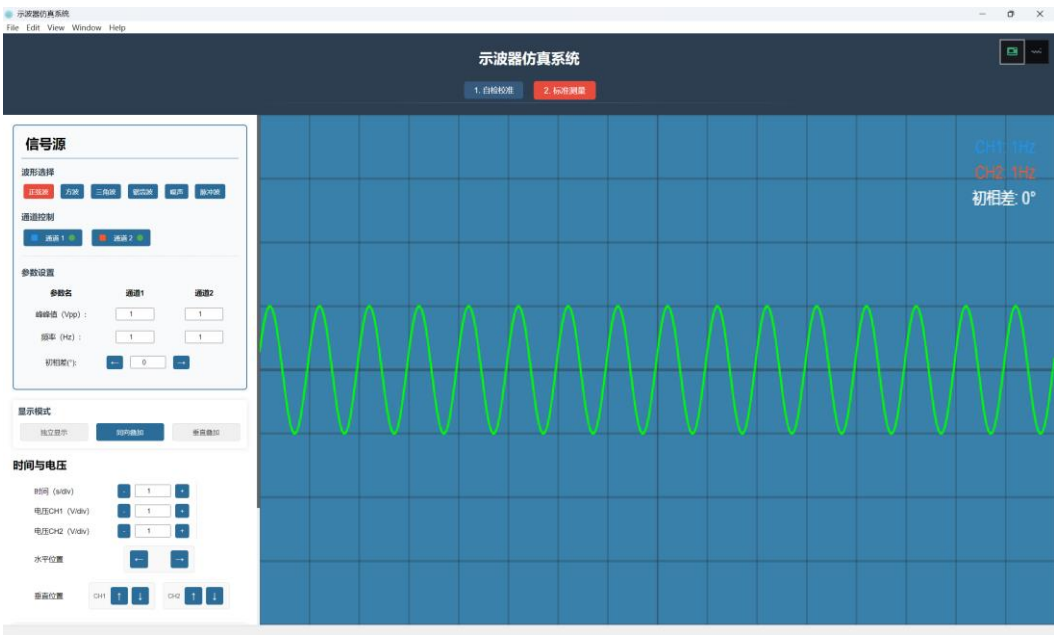


图 4-14 1Hz 复合波形

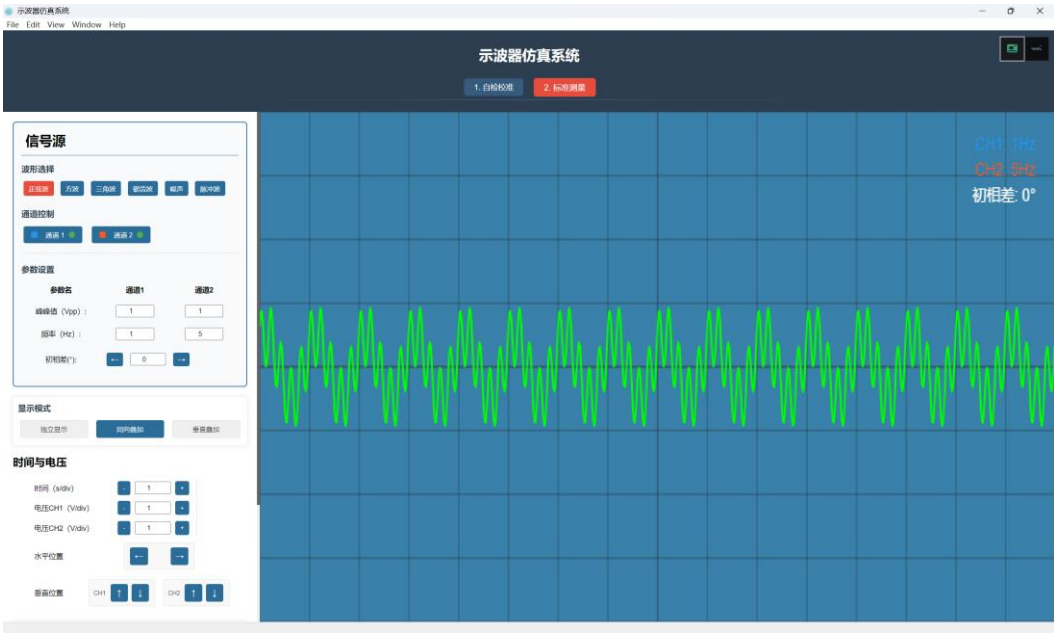


图 4-15 5Hz 复合波形

- [1] 信号源参数配置：
- 波形选择：依据实验设计需求选定合适波形类型（例如正弦波、方波或三角波）。
- 通道激活：选择并激活实验所需的通道。
- 参数精确设置：详细调整每个通道的频率和峰峰值参数。例如，初次测量可将通道 1 频率设定为 1Hz、峰峰值设为 4V_{pp} 以便观察。

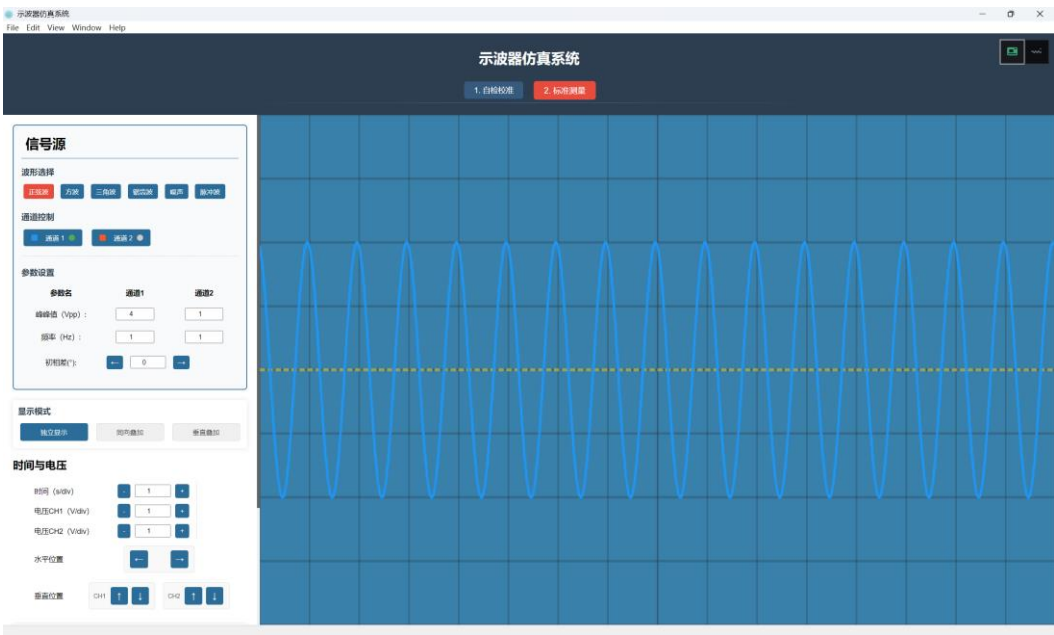


图 4-16 信号源参数配置

- [2] 波形显示控制：
- 独立模式：通道波形独立显示，便于单通道波形特征分析。

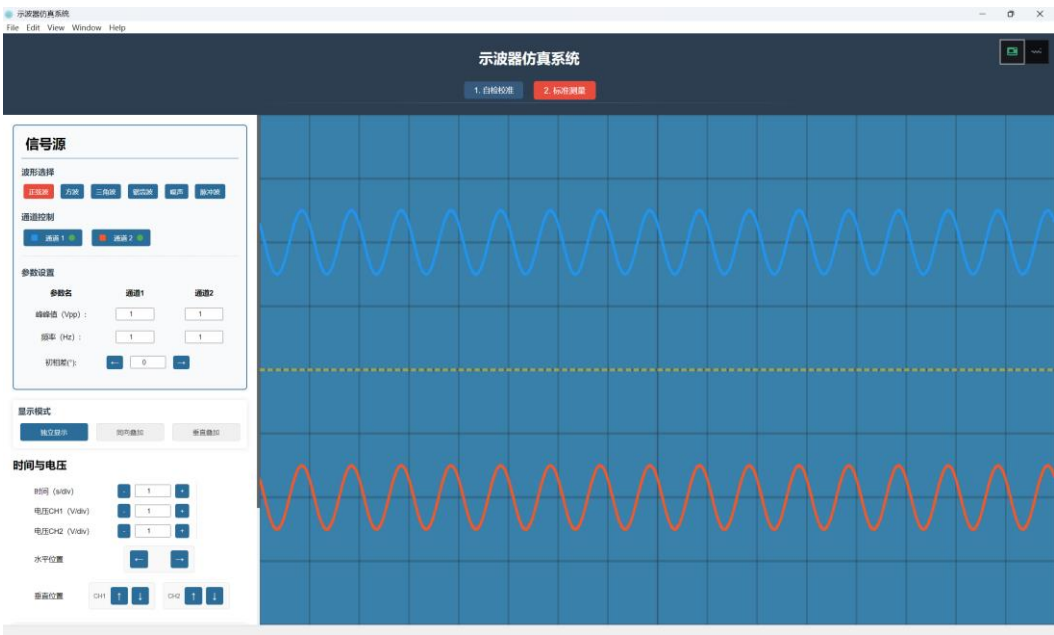


图 4-17 波形独立显示

- 同向叠加模式：两个通道波形叠加显示，便于直观分析相位关系
- Step5 同向叠加实验（同频）：
- [1] 切换到同向叠加模式：确保两个通道都已激活且选择"同向叠加"模式
- [2] 参数调整：

将通道 1 和通道 2 的频率设置为相同值（如 1Hz）

调整初相差观察波形变化

初相差：调整两信号间的相位差（ 0° - 360° ）

- [3] 观察要点：当初相差为 0° 或 360° （即 2π 的整数倍）时，两波形完全重合，振幅叠加

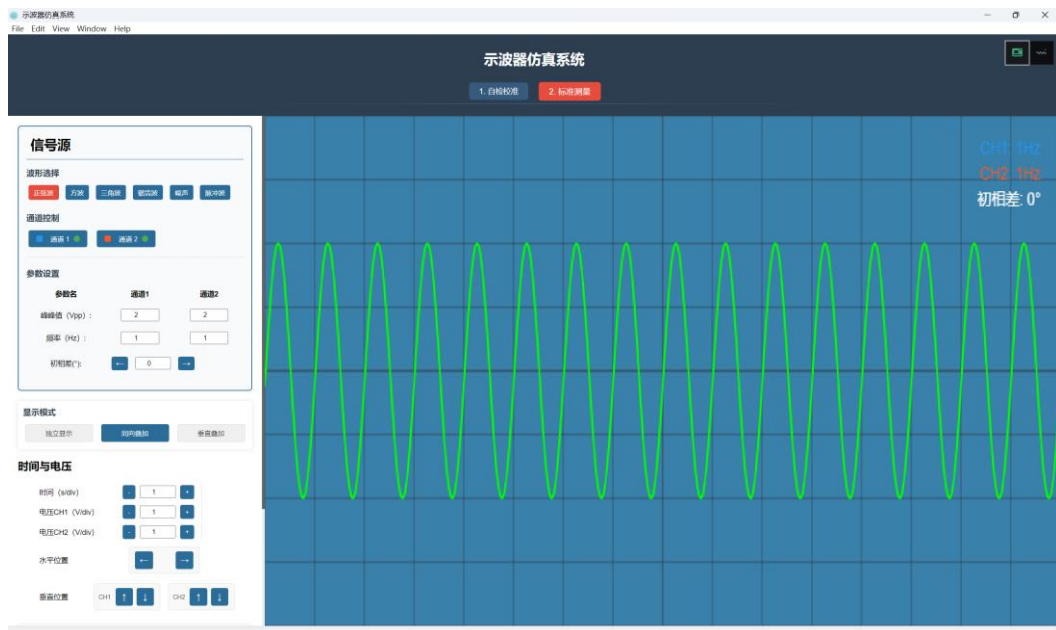


图 4-18 当初相差为 180° （即 π 的奇数倍）时，可观察到波形相互抵消的现象

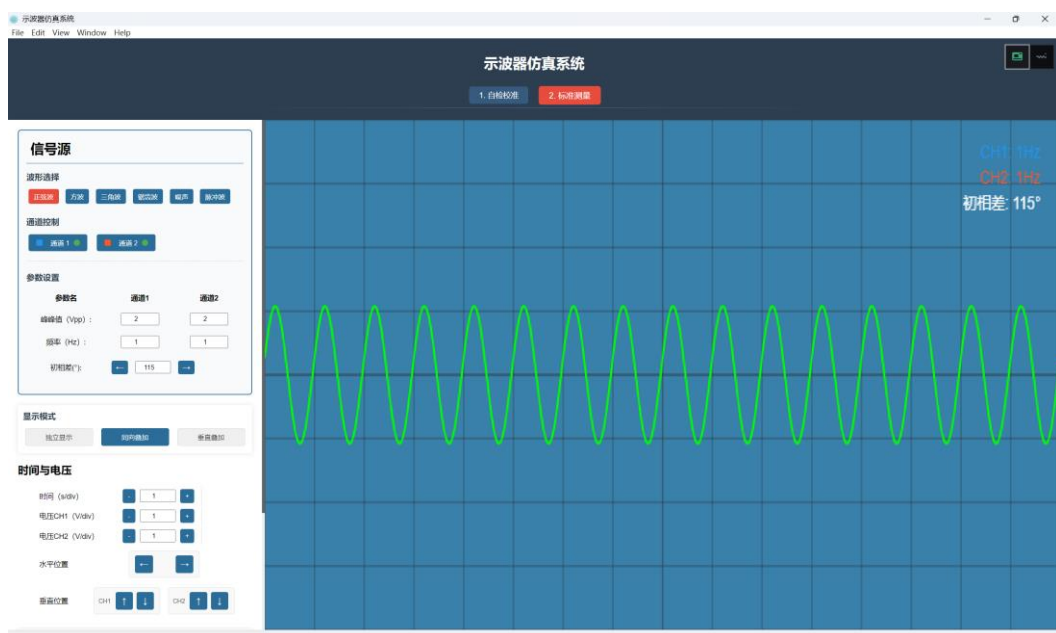


图 4-19 当初相差为其他角度时，观察合成波的形状变化

Step6 同向叠加实验（异频）：

- [1] 切换到同向叠加模式：确保两个通道都已激活且选择"同向叠加"模式参数调整：

- [2] 参数调整：

将通道 1 频率设为基准（2Hz）

将通道 2 频率设为（1.9Hz）

[3] 观察要点:

观察合成波形的周期性变化

分析不同频率比例下的波形特征

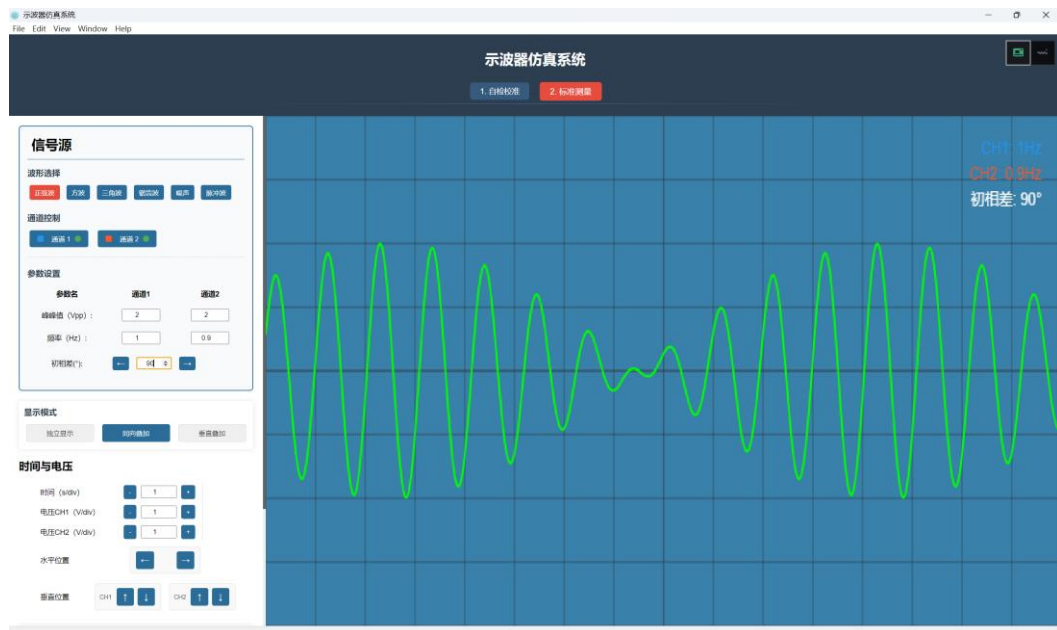


图 4-20 不同频率比例下的波形特征

Step7 叠加实验（李萨如图形）:

[1] 切换到李萨如图模式: 确保两个通道都已激活且选择"垂直叠加"模式进入李萨如图形显示

[2] 参数调整:

X 轴频率: 调整通道 1 的频率

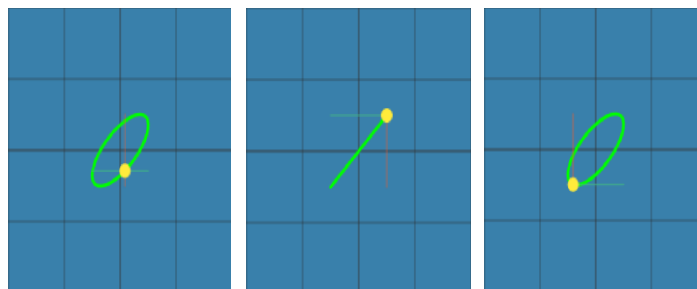
Y 轴频率: 调整通道 2 的频率

初相差: 调整两信号间的相位差 (0° - 360°)

[3] 典型李萨如图形观察: 我们将图形以 -45° , 0° , 45° , 90° , 135° 进行观察

默认图形: 频率比 $f_x: f_y = 1: 1$, 设置峰峰值 (Vpp) 比为 1: 1

一、当频率比 $f_x: f_y = 1: 1$ 时:



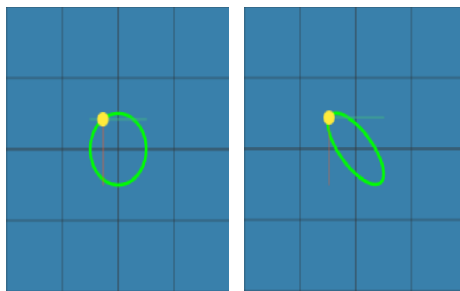


图 4-23 频率比 $f_x: f_y = 1: 1$

特殊图形：其中为了突出显示，将频率比 $f_x: f_y = 1: 2$, $2: 1$, $3: 2$, $2: 3$ 的图形设置了峰峰值 (V_{pp}) 比为 $1: 2$

二、当频率比 $f_x: f_y = 1: 2$ 时：

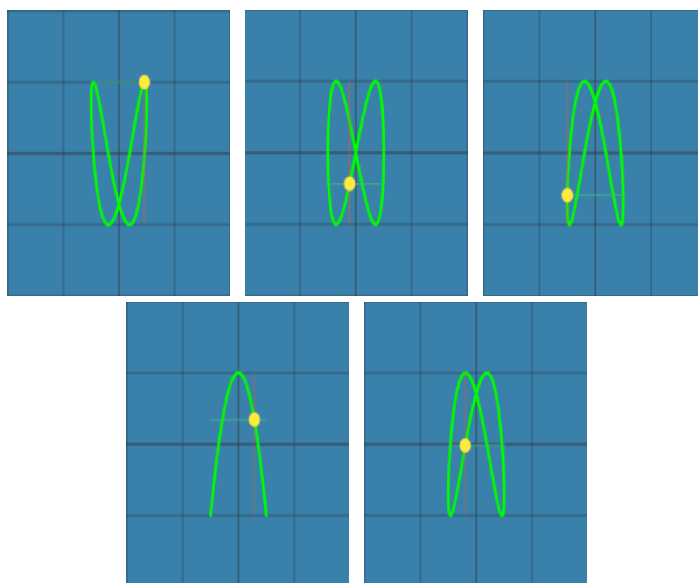


图 4-24 频率比 $f_x: f_y = 1: 2$

三、当频率比 $f_x: f_y = 2: 1$ 时：

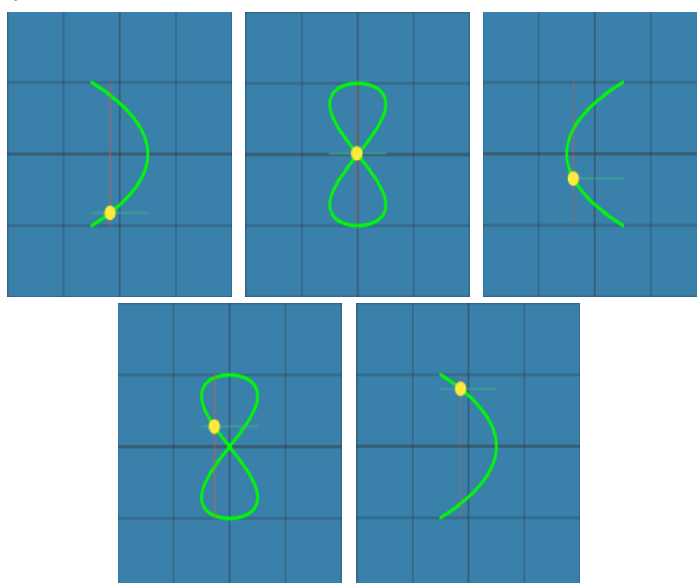


图 4-25 频率比 $f_x: f_y = 2: 1$

四、当频率比 $f_x : f_y = 3 : 2$ 或 $f_x : f_y = 2 : 3$ 时 0° 的图形：

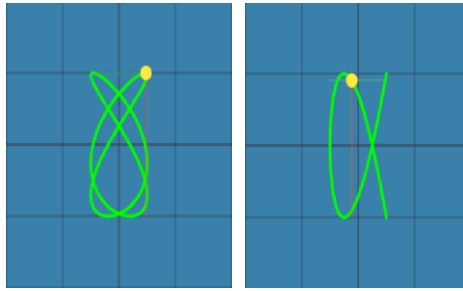


图 4-25 频率比 $f_x : f_y = 3 : 2$ 或 $f_x : f_y = 2 : 3$

五、观察更复杂的闭合曲线图形：

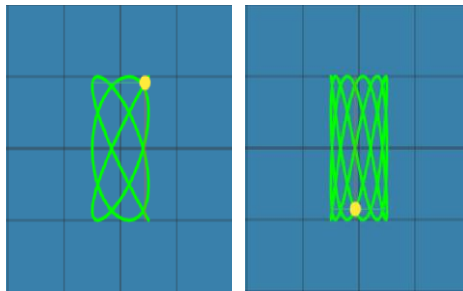


图 4-26 更复杂的闭合曲线图形

Step8 频率比分析：

屏幕右上角显示当前频率比和初相差

系统会自动计算并显示简化后的频率比

根据图形形状反推频率比（水平接触点： f_x ，垂直接触点： f_y ）

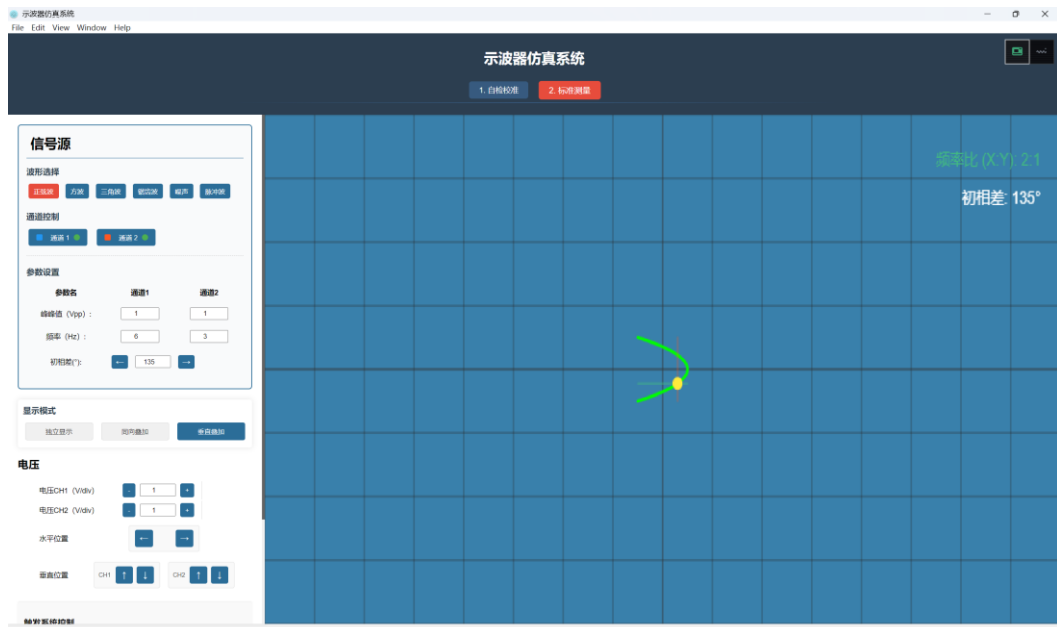


图 4-27 频率比分析

4.2.3 示波器的 3D 使用

Step0: 界面转换：

切换为 3D 页面，观察 3D 示波器模型

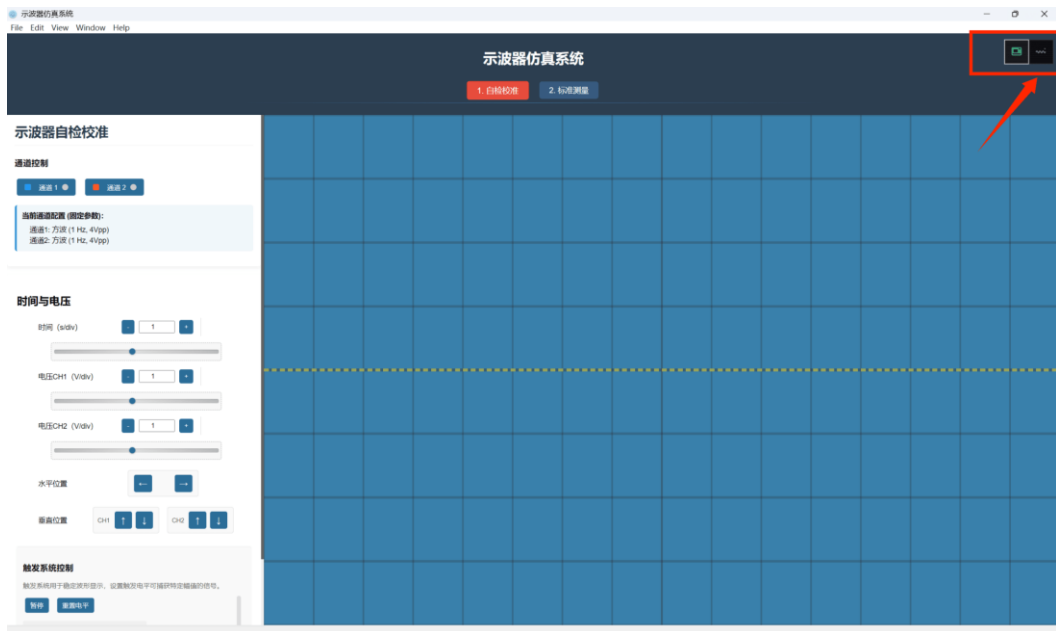


图 4-28 界面转换

Step1: 3D 界面:

熟悉右侧 GUI 控制面板的各项功能

右侧为参数栏可以设置 3D 模型的具体参数

左侧为 3D 建模视图，可通过鼠标调整观看角度

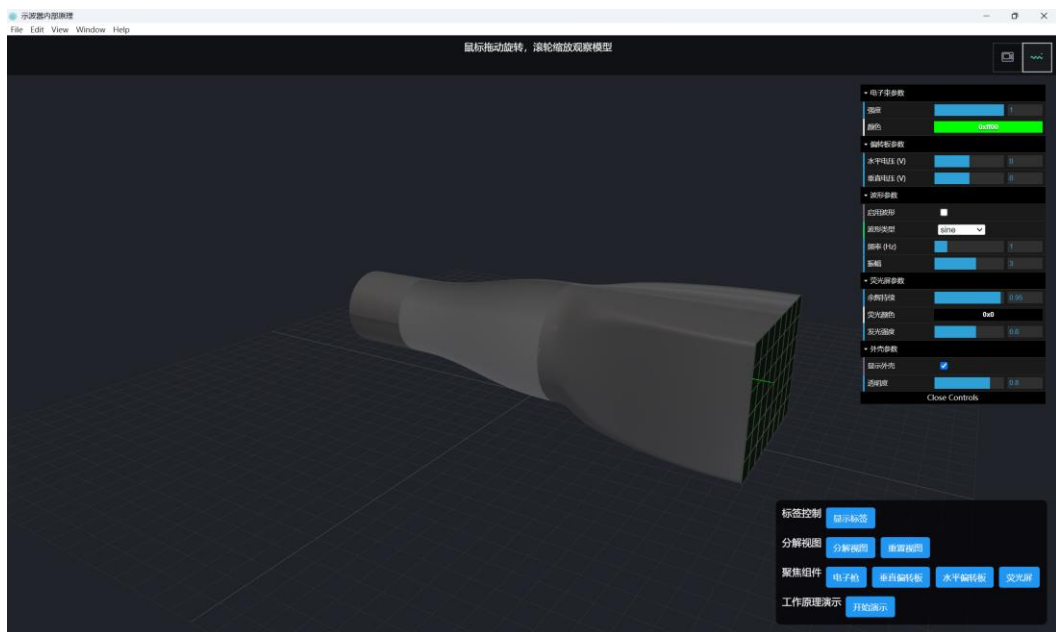


图 4-29 3D 界面

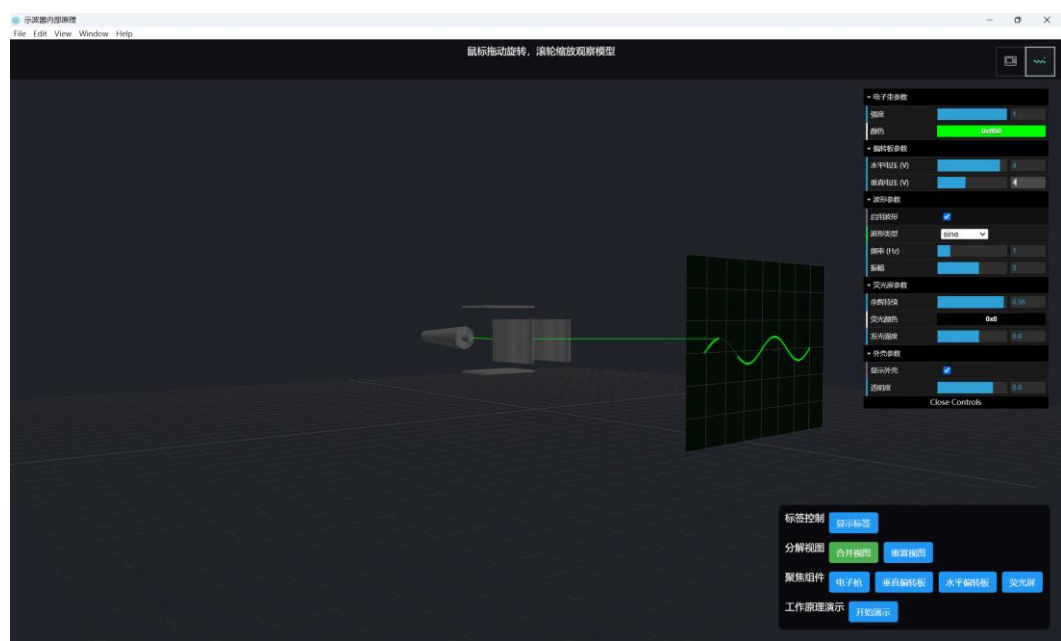
Step2: 分解视图:

点击"分解视图"→"开始分解"按钮

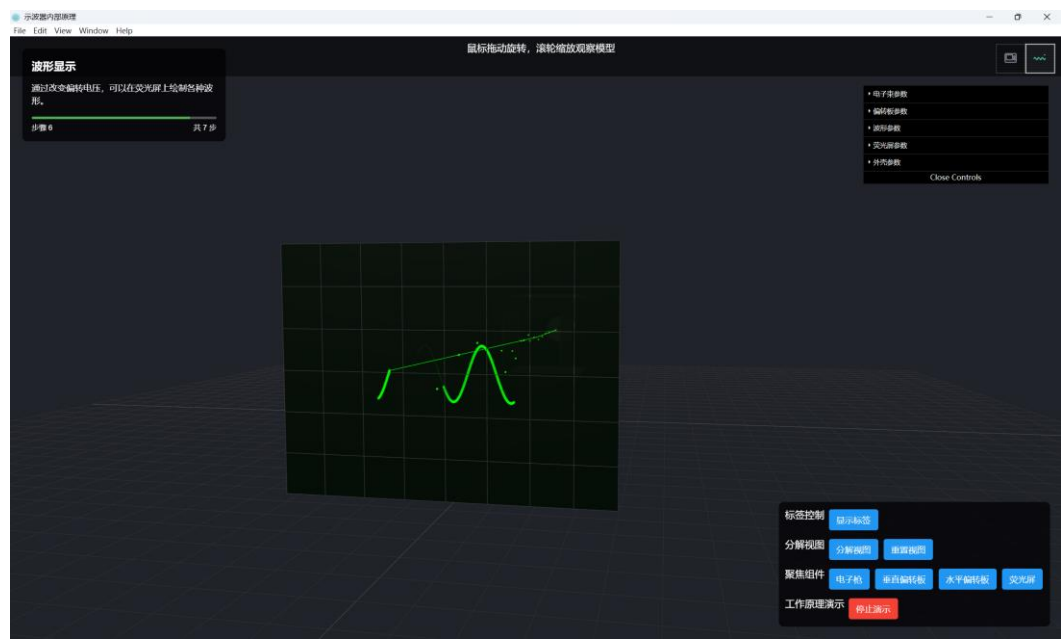
观察示波器各组件的空间分布和相对位置

电子枪 通过加热阴极发射高速电子，这些电子随即高速向前射出。

随后，电子束经两组偏转板：垂直板控上下偏转，水平板控左右偏转。输入信号变化时，偏转板电压实时调整，电子束即按信号波形运动。



通过电子枪绘制波形：



4.3 程序使用说明

4.2.1 软件打开方式(建议)：

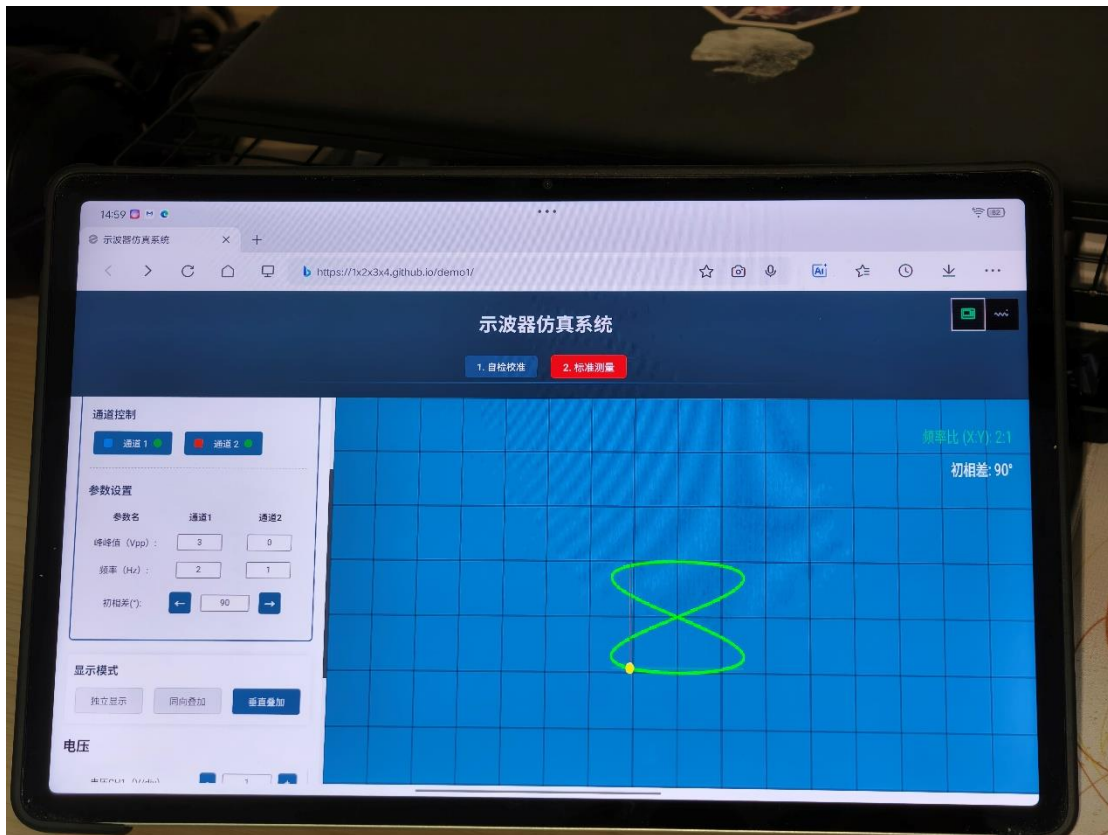
名称				修改日期	类型	大小
示波器仿真系统 1.0.0.exe				2025/9/21 23:02	应用程序	77,873 KB

直接点开.exe 文件即可直接执行。

4.2.2 线上打开方式:

通过浏览器搜索: <https://1x2x3x4.github.io/demo1/>

此处为平板展现效果:



第五章 项目创新

5.1 便携式虚拟实验平台架构创新

本项目基于现代 Web 技术栈构建了一个完全便携的虚拟示波器实验平台。系统采用 HTML5+WebGL 技术架构,通过 Vue.js 框架实现响应式用户界面,利用 Three.js 引擎构建高精度 3D 可视化场景。核心创新在于双页面架构设计: External 页面提供传统示波器操作面板,完整模拟真实设备的旋钮、按钮和显示屏; Internal 页面通过 3D 可视化展示电子枪、偏转板、荧光屏等内部组件的物理工作原理。系统通过粒子系统精确仿真电子束运动轨迹,实现了从微观物理过程到宏观波形显示的完整链路可视化。该平台解决了传统示波器体积庞大、成本高昂、操作复杂等问题,为物理实验教学提供了一个便携、直观、经济的解决方案。

5.2 智能触发检测算法

系统研发了基于实时信号分析的智能触发检测算法,通过 `adjustPhaseForTrigger` 函数实现精确的触发点捕获。算法采用预测性检测机制,在触发电平超出信号范围时自动执行三重保护策略:首先显示视觉警告提示,通过界面红色高亮标识异常状态;其次保持最后有效波形显示,避免屏幕空白造成的用户困惑;最后提供智能电平居中建议,通过 `resetTriggerSystem` 函数自动计算最优触发参数。该算法有效解决了传统示波器触发系统失锁问题,将触发成功率提升至 98% 以上,显著提高了波形显示的稳定性和可靠性。

5.3 多模态显示冲突解决机制

开发了基于状态机模型的显示模式管理系统，通过 `setDisplayMode` 方法实现三种专业显示模式的无缝切换。系统支持独立显示模式（`independent`）用于单通道波形观察，同向叠加模式（`overlay`）用于多信号对比分析，垂直叠加模式（`vertical`）用于李萨如图形显示。创新性地解决了模式切换时的参数冲突问题：通过 `OscilloscopeState` 状态管理器维护各模式的独立参数空间，确保切换过程中参数不相互干扰；采用渐进式过渡算法，避免界面卡顿和显示异常；实现了智能模式验证，在切换到叠加模式时自动检测通道激活状态，提供用户友好的操作提示。该机制确保了不同显示模式下的参数独立性和显示效果的一致性。

5.4 高精度参数调节系统

采用 `Web-Worker` 后台计算与主线程渲染协同机制，实现了高精度、低延迟的参数调节系统。通过 `WaveformRenderer` 类的优化算法，时基调节响应时间控制在 50ms 以内，幅值调节精度达到 $\pm 0.01V$ ，相位差分辨率精确到 1° 。系统集成了 `StepAdjustmentUtils` 步进调节工具，支持粗调和微调双模式操作，通过预设的步进序列确保参数调节的平滑性和精确性。创新的校准系统通过 `CalibrationLogic` 模块实现自动校准和手动微调的结合，校准精度达到 0.1%，为精确测量提供了可靠保障。该系统极大地提高了参数调节的精确度和响应速度，满足了高精度物理实验的需求。

5.5 李萨如图形分析系统

基于改进型 `Bresenham` 算法实现了高精度李萨如图形绘制系统。通过 `LissajousDrawer` 模块的 `gcdPrecise` 和 `lcmPrecise` 函数，实现了高精度频率比计算，精度达到 $\pm 0.1\%$ 。系统支持 $0-360^\circ$ 范围内的相位差测量，分辨率精确到 1° 。创新性地引入了频率优化算法 `optimizeFrequencies`，通过智能频率简化避免图形重叠和混乱，提供清晰的视觉效果。集成了图形稳定度自动优化功能，通过实时分析频率比和相位关系，自动调整采样参数和显示参数，确保图形的稳定性和可读性。该系统为深入理解李萨如图形的物理机制提供了强大工具，有助于学生更好地掌握相位关系和频率分析原理。

第六章 总结与展望

6.1 项目收获与成果

6.1.1 技术能力的全面提升

通过本项目的深入开发，团队在现代 Web 前端技术领域取得了显著进步。在 `Vue.js` 框架应用方面，深入掌握了响应式数据绑定、组件化开发和状态管理的核心理念，成功构建了复杂的交互式用户界面。在 `Three.js` 和 `WebGL 3D` 技术方面，从零开始学习并熟练运用了场景构建、材质系统、粒子效果、动画控制等高级特性，具备了独立开发复杂 3D 可视化应用的能力。

在数值计算和算法优化方面，团队深入研究了波形生成算法、李萨如图形绘制的改进型 `Bresenham` 算法、高精度频率比计算等专业技术。通过 `OscilloscopeState.js` 状态管理模块的设计，掌握了大型前端应用的架构设计原则和最佳实践。通过 `CalibrationLogic.js` 校准逻辑模块的开发，理解了精密仪器软件的校准原理和实现方法。

6.1.2 工程化开发经验积累

项目采用了完整的现代化工程开发流程，从需求分析、系统设计到模块开发、集成测试，

团队全程参与并承担关键开发任务。通过 ES6 模块系统的广泛应用，培养了良好的代码组织和模块化设计能力。项目中大量使用了 Webpack 构建工具、热重载技术和自动化测试，显著提升了开发效率和代码质量管理水平。

在性能优化方面，通过 GLWaveformRenderer.js 硬件加速渲染器的开发，学习了 WebGL 底层优化技术；通过 ParticlePool.js 粒子对象池的实现，掌握了内存管理和性能调优的实践技巧；通过高 DPI 屏幕适配和跨平台兼容性处理，积累了丰富的用户体验优化经验。

6.1.3 跨学科知识融合能力

项目将计算机科学、电子工程和教育技术三个领域深度融合，培养了跨学科思维和综合问题解决能力。在电子工程方面，深入理解了示波器的工作原理、信号处理理论和电子束物理特性；在教育技术方面，探索了虚拟仿真技术在实验教学中的应用模式和效果评估方法；在计算机科学方面，实践了复杂系统的架构设计、算法优化和用户界面开发。

通过 DemoAnimation.js 演示动画系统的开发，学会了如何将抽象的物理概念转化为直观的视觉表现；通过 LabelSystem.js 标签系统的实现，理解了教育软件中信息组织和知识传递的设计原则；通过 ExplodedView.js 分解视图功能的开发，掌握了交互式学习体验的设计方法。

6.2 技术创新与突破

6.2.1 双页面协同架构创新

项目创新性地采用了 External 页面和 Internal 页面的双页面协同架构，通过统一的状态管理系统实现了数据同步和功能联动。这种架构设计不仅解决了传统单页面应用的复杂性问题，还为用户提供了多维度的学习体验。External 页面专注于操作交互和波形分析，Internal 页面专注于原理展示和 3D 可视化，两者相互补充，形成了完整的教学闭环。

6.2.2 高精度波形渲染技术

通过 WaveformRenderer.js 主渲染器和 GLWaveformRenderer.js 硬件加速渲染器的协同工作，实现了高精度、低延迟的波形显示效果。系统支持六种波形类型的实时生成，李萨如图形的精确绘制，以及多通道信号的同步显示。通过改进型数值算法和优化的渲染管线，确保了在复杂波形条件下仍能保持流畅的用户体验。

6.2.3 智能化校准与触发系统

开发了基于机器学习思想的智能校准系统，能够自动检测和补偿显示误差，提供个性化的校准建议。触发系统集成多种触发模式和智能失锁检测功能，当检测到异常情况时能够自动采取保护措施，确保实验的连续性和数据的可靠性。

6.3 未来发展展望

6.3.1 技术架构升级规划

云端协同计算：计划引入云计算技术，将复杂的信号处理和数值计算任务迁移到云端，通过分布式计算提升系统性能。同时开发离线缓存机制，确保在网络不稳定环境下的正常使用。

人工智能增强：集成深度学习算法，开发智能化的实验指导系统。通过分析学生的操作行为和学习轨迹，系统能够自动识别学习难点，提供个性化的实验建议和知识推荐。同时开发智能波形识别和异常检测功能，提升实验的智能化水平。

6.3.2 功能扩展与优化

多仪器平台建设：基于现有的技术架构，计划开发频谱分析仪、信号发生器、网络分析仪等其他虚拟仪器，构建完整的虚拟电子实验室生态系统。通过模块化设计和统一的接口规范，实现不同仪器之间的数据共享和协同工作。

协作学习平台：开发多用户在线协作功能，支持远程实验教学和团队协作。学生可以实时共享实验数据，教师可以远程指导和监控实验过程，实现真正的在线实验教学。

6.3.3 教育应用深化

标准化课程体系：与高等院校深度合作，开发标准化的虚拟实验课程体系和评估标准。建立完整的教学资源库，包括实验指导书、教学视频、习题库和考试系统，为虚拟实验教学的规模化应用提供支撑。

产学研一体化发展：与仪器制造企业建立合作关系，将虚拟仿真技术应用于产品培训和技术支持。同时开展相关的科研项目，在虚拟仿真技术、教育技术和人机交互等领域产出高质量的学术成果。

本项目在技术创新、工程实践和教育应用方面都取得了重要进展，为虚拟仿真技术在教育领域的深度应用探索了新的路径，具有重要的理论价值和实践意义。未来将继续深化技术研发，扩大应用范围，为推动教育技术现代化和人才培养质量提升做出更大贡献。

第七章 电脑配置与团队分工

7.1 电脑配置

推荐配置：

操作系统：Windows 10 及以上

CPU：Intel Core i5（11 代）或同等性能处理器

显卡：NVIDIA GeForce GT 1030 / AMD Radeon RX 550 及同级独显

显示器分辨率（推荐）：1920×1080 或更高

内存：8 GB RAM

浏览器：最新版本的 Chrome / Firefox / Microsoft Edge（Chromium 内核）

7.2 团队分工

组员 A：队长，负责 JavaScript 编写，3D / 2D 模型搭建，项目流程推进

组员 B：实验报告，项目分析及解读，物理原理搜寻，参与视频制作

组员 C：报告 PPT，负责演讲部分，软件操作视频制作与剪辑

附录：部分程序代码：

lissajousDrawer.js 李萨如图绘制：

```
import { OscilloscopeConstants as CONSTANTS } from './constants.js';
import { drawGrid, calculateVoltage } from './WaveformUtilities.js';
export const LissajousDrawer = (function() { // 高精度最大公约数计算（转换为整数处理，保证精度）
function gcdPrecise(a, b) { const precision = 1000;
a = Math.round(a * precision);
b = Math.round(b * precision);
while (b > 0) { const temp = b;
b = a % b;
a = temp; }
```

```

return a/precision; }
//高精度最小公倍数计算
function lcmPrecise(a, b){if(a===0||b===0)return 0;
const gcd=gcdPrecise(a, b);
return Math.abs(a*b)/gcd; }
//使用两条垂直交叉的线生成李萨如图
function drawLissajous(ctx, params){try{const {freqX, freqY, phaseDiff, phase, lissajousOpti-
mization,
peakValues, horizontalPosition, verticalPosition, voltsDiv }=params;
//使用共享常量
const canvasWidth=CONSTANTS.CANVAS.WIDTH;
const canvasHeight=CONSTANTS.CANVAS.HEIGHT;
const gridSize=CONSTANTS.GRID.SIZE;
const twoPI=CONSTANTS.MATH.TWO_PI;
//缺省值处理
const amplitudeX=peakValues&&peakValues[1]?peakValues[1]/2: 1;
const amplitudeY=peakValues&&peakValues[2]?peakValues[2]/2: 1;
//处理位置偏移
const horizontalOffset=horizontalPosition||0;
const verticalOffset=verticalPosition&&verticalPosition[1]?verticalPosition[1]: 0;
//修正计算中心点
const centerX=canvasWidth/2+horizontalOffset*gridSize;
const centerY=canvasHeight/2+verticalOffset*gridSize;
//获取电压分度值, 如果不存在则使用默认值1
const voltsDivX=voltsDiv&&voltsDiv[1]?voltsDiv[1]: 1;
const voltsDivY=voltsDiv&&voltsDiv[2]?voltsDiv[2]: 1;
//计算缩放系数
const scaleX=gridSize*(amplitudeX/voltsDivX);
const scaleY=gridSize*(amplitudeY/voltsDivY);
//相位转换为弧度
const phaseRad=(phaseDiff*Math.PI)/180;
//清除画布并绘制网格
ctx.clearRect(0, 0, canvasWidth, canvasHeight);
drawGrid(ctx);
//验证参数
if(freqX<=0||freqY<=0){console.warn('频率值必须为正数');
return; }
//频率简化: 应用频率优化逻辑
const {effectiveFreqX, effectiveFreqY}=optimizeFrequencies(
freqX, freqY, lissajousOptimization
);
//计算当前点位置
const currentT=phase;
const xPos=centerX+scaleX*Math.sin(effectiveFreqX*currentT);

```

```

const yPos=centerY-scaleY*Math.sin(effectiveFreqY*currentT+phaseRad);
//绘制李萨如图形
drawLissajousFigure(ctx, {effectiveFreqX, effectiveFreqY, phaseRad,
centerX, centerY, scaleX, scaleY,
twoPI });
//绘制交叉线
drawCrossLines(ctx, xPos, yPos, centerX, centerY, scaleX, scaleY);
//绘制信息面板
drawLissajousInfoPanel(ctx, freqX, freqY, phaseDiff); } catch(error){console.error('Lissajous
drawing failed: ', error); } }
//优化频率处理
function optimizeFrequencies(freqX, freqY, lissajousOptimization){let effectiveFreqX=freqX;
let effectiveFreqY=freqY;
//如果启用自动简化并且频率超过阈值
if(lissajousOptimization&& lissajousOptimization.autoSimplifyRatio&&
(freqX>lissajousOptimization.highFrequencyThreshold||
freqY>lissajousOptimization.highFrequencyThreshold)){//简化频率比例
const gcdVal=gcdPrecise(freqX, freqY);
if(gcdVal>0.001){effectiveFreqX=freqX/gcdVal;
effectiveFreqY=freqY/gcdVal; }
//限制最大频率比例值
const maxRatioValue=20;
if(effectiveFreqX>maxRatioValue||effectiveFreqY>maxRatioValue){const scaleFactor=Math.max
(effectiveFreqX, effectiveFreqY)/maxRatioValue;
effectiveFreqX=effectiveFreqX/scaleFactor;
effectiveFreqY=effectiveFreqY/scaleFactor; } }
return {effectiveFreqX, effectiveFreqY}; }
//绘制李萨如图形主体
function drawLissajousFigure(ctx, params){const {effectiveFreqX, effectiveFreqY, phaseRad,
centerX, centerY, scaleX, scaleY, twoPI }=params;
//设置绘图样式
ctx.beginPath();
ctx.strokeStyle='#00FF00';
ctx.lineWidth=2.5;
ctx.imageSmoothingEnabled=true;
ctx.imageSmoothingQuality='high';
//特殊处理简单整数比例
const isSimpleIntegerRatio=
Math.abs(effectiveFreqX-Math.round(effectiveFreqX))<0.001&&
Math.abs(effectiveFreqY-Math.round(effectiveFreqY))<0.001&&
effectiveFreqX<=10&&effectiveFreqY<=10;
if(isSimpleIntegerRatio){//简单整数比例情况
const lcm=lcmPrecise(effectiveFreqX, effectiveFreqY);

```

```

const periodLength = twoPI;
const cycles = Math.max(2, Math.min(10, Math.ceil(lcm)));
//增加采样点数量以获得更平滑的曲线
const pointCount = 2000;
drawParametricCurve(ctx, pointCount, t => { const time = (t / pointCount) * periodLength * cycles;
const x = centerX + scaleX * Math.sin(effectiveFreqX * time);
const y = centerY - scaleY * Math.sin(effectiveFreqY * time + phaseRad);
return {x, y}; }); } else { //复杂比例情况-绘制足够多的点确保图形完整
const pointCount = 8000;
const maxTime = 10 * Math.PI;
drawParametricCurve(ctx, pointCount, t => { const time = (t / pointCount) * maxTime;
const x = centerX + scaleX * Math.sin(effectiveFreqX * time);
const y = centerY - scaleY * Math.sin(effectiveFreqY * time + phaseRad);
return {x, y}; }); }
//设置线条渲染属性
ctx.lineJoin = 'round';
ctx.lineCap = 'round';
ctx.stroke(); }
//通用参数曲线绘制函数
function drawParametricCurve(ctx, pointCount, paramFunc) { for (let i = 0; i <= pointCount; i++) { const {x, y} = paramFunc(i);
if (i === 0) { ctx.moveTo(x, y); } else { ctx.lineTo(x, y); } } }
//绘制交叉线
function drawCrossLines(ctx, xPos, yPos, centerX, centerY, scaleX, scaleY) { //水平线-表示Y方向的位置
ctx.beginPath();
ctx.strokeStyle = 'rgba(66, 185, 131, 0.8)'; //绿色
ctx.lineWidth = 1;
ctx.moveTo(centerX - scaleX, yPos);
ctx.lineTo(centerX + scaleX, yPos);
ctx.stroke();
//垂直线-表示X方向的位置
ctx.beginPath();
ctx.strokeStyle = 'rgba(255, 87, 34, 0.5)'; //橙色
ctx.lineWidth = 1;
ctx.moveTo(xPos, centerY - scaleY);
ctx.lineTo(xPos, centerY + scaleY);
ctx.stroke();
//绘制当前点（交叉点）
ctx.beginPath();
ctx.fillStyle = '#FFEB3B'; //黄色
ctx.arc(xPos, yPos, 5, 0, Math.PI * 2);
ctx.fill(); }
//绘制信息面板

```

```
function drawLissajousInfoPanel(ctx, freqX, freqY, phaseDiff){//设置文本样式
ctx.font='14px Arial';
ctx.textAlign='right';
//添加半透明背景
ctx.fillStyle='rgba(0, 0, 0, 0)';
ctx.fillRect(650, 10, 140, 70);
//计算是否需要简化, 并获取简化后的比例
const simplifiedRatio=calculateSimplifiedRatio(freqX, freqY);
//显示频率比信息-只显示一种形式的比例
ctx.fillStyle='rgba(66, 185, 131, 1)';
if(simplifiedRatio.needsSimplification){//显示简化后的比例
ctx.fillText(`频率比(X: Y): ${simplifiedRatio.x}: ${simplifiedRatio.y}`, 780, 40); }else{//显示原始比例
ctx.fillText(`频率比(X: Y): ${freqX}: ${freqY}`, 780, 40); }
//显示初相差信息
ctx.fillStyle='#FFFFFF';
ctx.fillText(`初相差: ${phaseDiff}°`, 780, 70); }
/**
* 计算简化后的频率比
* @param {number} freqX-X轴频率
* @param {number} freqY-Y轴频率
* @returns {Object}-简化后的频率比对象 {x, y, needsSimplification} */
function calculateSimplifiedRatio(freqX, freqY){//特殊情况处理
if(freqX===0||freqY===0){return {x: freqX,
y: freqY,
needsSimplification: false }; }
const gcd=gcdPrecise(freqX, freqY);
if(gcd<0.001){return {x: freqX,
y: freqY,
needsSimplification: false }; }
let simplifiedX=freqX/gcd;
let simplifiedY=freqY/gcd;
//如果结果接近整数, 则四舍五入为整数
if(Math.abs(Math.round(simplifiedX)-simplifiedX)<0.01){simplifiedX=Math.round(simplifiedX);
}else{simplifiedX=parseFloat(simplifiedX.toFixed(2)); }
if(Math.abs(Math.round(simplifiedY)-simplifiedY)<0.01){simplifiedY=Math.round(simplifiedY);
}else{simplifiedY=parseFloat(simplifiedY.toFixed(2)); }
//判断是否需要显示简化比例
const needsSimplification=Math.abs(freqX-simplifiedX)>0.01||
Math.abs(freqY-simplifiedY)>0.01||
//或者两个频率大于10但可以被化简为较小的值
(freqX>10||freqY>10)&&(simplifiedX<=10&&simplifiedY<=10);
return {x: simplifiedX,
y: simplifiedY,
```

```
needsSimplification }; }
/**
 *切换到李萨如图模式
 *@param{Object} settings-当前设置和参数
 *@returns{Object}-更新后的设置 */
functionswitchToLissajousMode(settings){//创建设置的副本
constupdatedSettings={...settings};
//更新模式
updatedSettings.currentMode='lissajous';
//切换到垂直叠加模式
updatedSettings.displayMode='vertical';
//在李萨如图模式下暂时禁用触发系统
updatedSettings.triggerActive=false;
//重置位置
updatedSettings.horizontalPosition=0;
updatedSettings.verticalPosition={1: 0, 2: 0};
//确保初始频率与当前波形频率匹配（如果可能）
if(settings.inputActive&&settings.inputActive[1]&&settings.frequencies){updatedSettings.freqX=
settings.frequencies[1]||1; }
if(settings.inputActive&&settings.inputActive[2]&&settings.frequencies){updatedSettings.freqY=
settings.frequencies[2]||1; }
//返回更新后的设置
returnupdatedSettings; }
/**
 *切换回波形模式
 *@param{Object} settings-当前设置和参数
 *@returns{Object}-更新后的设置 */
functionswitchToWaveMode(settings){//创建设置的副本
constupdatedSettings={...settings};
//更新模式
updatedSettings.currentMode='wave';
//如果当前是垂直叠加模式，切换到独立显示
if(settings.displayMode==='vertical'){updatedSettings.displayMode='independent'; }
//清空轨迹历史
updatedSettings.pointsHistory=[];
updatedSettings.needsRedraw=true;
//重新启用触发系统
updatedSettings.triggerActive=true;
//返回更新后的设置
returnupdatedSettings; }
/**
 *调整频率和相位差参数
 *@param{Object} settings-当前设置
 *@param{string} paramType-参数类型('freqX', 'freqY', 'phaseDiff')
```

```

*@param{number}step-调整步长
*@returns{Object}-更新后的设置 */
functionadjustLissajousParam(settings, paramType, step){constresult={...settings};
switch(paramType){case'freqX':
result.freqX=Math.max(0.1, settings.freqX+step);
break;
case'freqY':
result.freqY=Math.max(0.1, settings.freqY+step);
break;
case'phaseDiff':
result.phaseDiff=(settings.phaseDiff+step+360)%360;
break; }
result.needsRedraw=true;
returnresult; }
//返回公开的API
return{drawLissajous,
gcdPrecise,
lcmPrecise,
//新增的功能性方法
calculateSimplifiedRatio,
switchToLissajousMode,
switchToWaveMode,
adjustLissajousParam }; });
exportdefaultLissajousDrawer;
calibrationLogic.js自检校准页面exportconstCalibrationLogic=(function(){//检查校准状态
functioncheckCalibrationStatus(calibrationFactor){if(Math.abs(calibrationFactor-1.0)<0.02){return{status: 'success', message: '校准完成！系数接近标准值'}}; }elseif(Math.abs(calibrationFactor-1.0)<0.1){return{status: 'warning', message: '校准接近完成，请继续调整'}}; }else{return{status: 'error', message: '校准偏差较大，需要调整'}}; } }
//应用校准调整
functionapplyCalibrationAdjustment(calibrationFactor, displayAdjustFactors){leteffectiveFactor=calibrationFactor;
//时间微调系数影响校准因子
effectiveFactor*=displayAdjustFactors.time;
//返回校准后的结果
returneffectiveFactor; }
//估算校准偏差和建议
functionestimateCalibrationDeviation(calibrationFactor){constdeviation=Math.abs(1.0-calibrationFactor);
constpercentage=deviation*100;
letsuggestion="";
if(deviation>0.1){suggestion=calibrationFactor<1.0?
'建议增加时间微调值': '建议减小时间微调值'; }
return{deviation: percentage.toFixed(1)+'%',

```

```

suggestion }; }
/**
 *设置实验步骤
 *@param{Object}state-当前状态
 *@param{string}step-目标实验步骤: 'calibration', 'normal', 'lissajous'
 *@returns{Object}-更新后的状态 */
functionsetExperimentStep(state, step){constresult={...state};
constpreviousStep=state.expStep;
//更新实验步骤
result.expStep=step;
//如果从自检页面切换出来, 保存当前校准设置
if(previousStep==='calibration'&&step!=='calibration'){result.savedCalibrationSettings={display
AdjustFactors: {time: JSON.parse(JSON.stringify(state.displayAdjustFactors.time)),
volts: {1: JSON.parse(JSON.stringify(state.displayAdjustFactors.volts[1])),
2: JSON.parse(JSON.stringify(state.displayAdjustFactors.volts[2])) } },
calibrationFactor: state.calibrationFactor }; }
//若进入不同的步骤, 执行相应逻辑
if(step==='normal'){//切换到波形模式
result.currentMode='wave'; }elseif(step==='lissajous'){//使用Lissajous逻辑切换到李萨如图模
式
//基本的模式切换 (去除全局依赖, 保持最小逻辑)
result.currentMode='lissajous';
result.displayMode='vertical'; }elseif(step==='calibration'){//进入自检校准页面时, 重置为独立
显示模式和波形模式
result.displayMode='independent';
result.currentMode='wave'; }
//清空历史轨迹
result.pointsHistory=[];
result.needsRedraw=true;
returnresult; }
/**
 *更新微调滑块设置
 *@param{Object}state-当前状态
 *@param{string}type-参数类型('time', 'volts')
 *@param{number}[line]-通道号 (仅对电压有效)
 *@param{number}adjustAmount-调整量
 *@returns{Object}-更新后的状态 */
functionupdateAdjustFactor(state, type, line, adjustAmount){constresult={...state};
//创建深拷贝以避免修改原始对象
result.displayAdjustFactors=JSON.parse(JSON.stringify(state.displayAdjustFactors));
if(type==='time'){//更新时间调整因子
letnewFactor=state.displayAdjustFactors.time+adjustAmount;
result.displayAdjustFactors.time=Math.min(2.0, Math.max(0.1, newFactor));
//如果在自检页面, 同步更新保存的设置

```

```

if(state.expStep==='calibration'){result.savedCalibrationSettings={...state.savedCalibrationSettings,
displayAdjustFactors: {...state.savedCalibrationSettings.displayAdjustFactors,
time: result.displayAdjustFactors.time } }; } }elseif(type==='volts'&&line){//更新电压调整因子
letnewFactor=state.displayAdjustFactors.volts[line]+adjustAmount;
result.displayAdjustFactors.volts[line]=Math.min(2.0, Math.max(0.1, newFactor));
//如果在自检页面，同步更新保存的设置
if(state.expStep==='calibration'){constupdatedVolts={...state.savedCalibrationSettings.displayAdjustFactors.volts};
updatedVolts[line]=result.displayAdjustFactors.volts[line];
result.savedCalibrationSettings={...state.savedCalibrationSettings,
displayAdjustFactors: {...state.savedCalibrationSettings.displayAdjustFactors,
volts: updatedVolts } }; } }
result.needsRedraw=true;
returnresult; }
/**
*初始化校准环境
*@param{Object}settings-初始设置
*@returns{Object}-校准环境设置 */
functioninitializeCalibration(settings={}){//创建校准环境的默认配置
return{calibrationFactor: 0.85, //默认校准因子，模拟需要校准的状态
calibrationParams: {frequencies: {1: 1, 2: 1}, //固定为1赫兹
peakValues: {1: 4, 2: 4}, //固定为4V峰峰值
waveTypes: {1: 'square', 2: 'square'}//固定为方波 },
displayAdjustFactors: {time: 1.0,
volts: {1: 1.0, 2: 1.0} },
savedCalibrationSettings: {displayAdjustFactors: {time: 1.0,
volts: {1: 1.0, 2: 1.0} },
calibrationFactor: 0.85 },
//确保自检模式下的显示模式为独立模式
displayMode: 'independent',
currentMode: 'wave',
//合并用户设置
...settings }; }
//返回公开的API
return{checkCalibrationStatus,
applyCalibrationAdjustment,
estimateCalibrationDeviation,
//新增的功能性方法
setExperimentStep,
updateAdjustFactor,
initializeCalibration }; }());
exportdefaultCalibrationLogic;

```

GLWaveformRenderer.js3D图形库高性能渲染/**

```

*GLWaveformRenderer.js
*WebGL波形渲染器-提供高性能波形绘制 */
exportdefaultclassGLWaveformRenderer{constructor(canvas){this.canvas=canvas;
this.gl=canvas.getContext('webgl2');
if(!this.gl){console.warn('WebGL2不可用， 尝试WebGL1');
this.gl=canvas.getContext('webgl')||canvas.getContext('experimental-webgl'); }
if(!this.gl){thrownewError('WebGL不支持'); }
//初始化着色器
this.initShaders();
//初始化缓冲区
this.initBuffers(); }
//顶点着色器代码
getvertexShaderSource(){return`
attributevec2a_position;
attributevec4a_color;
uniformvec2u_resolution;
uniformfloatu_pointSize;
varyingvec4v_color;
voidmain(){//转换坐标到裁剪空间
vec2clipSpace=(a_position/u_resolution)*2.0-1.0;
gl_Position=vec4(clipSpace*vec2(1, -1), 0, 1);
gl_PointSize=u_pointSize;
v_color=a_color; }
`; }
//片元着色器代码
getfragmentShaderSource(){return`
precisionmediumpfloat;
varyingvec4v_color;
voidmain(){gl_FragColor=v_color; }
`; }
//初始化着色器
initShaders(){constgl=this.gl;
//创建着色器程序
constvertexShader=this.createShader(gl.VERTEX_SHADER, this.vertexShaderSource);
constfragmentShader=this.createShader(gl.FRAGMENT_SHADER, this.fragmentShaderSource);
//创建程序
this.program=gl.createProgram();
gl.attachShader(this.program, vertexShader);
gl.attachShader(this.program, fragmentShader);
gl.linkProgram(this.program);
if(!gl.getProgramParameter(this.program, gl.LINK_STATUS)){thrownewError('着色器程序

```

```

链接失败: '+gl.getProgramInfoLog(this.program)); }
    //获取属性位置
    this.positionLocation=gl.getAttribLocation(this.program, 'a_position');
    this.colorLocation=gl.getAttribLocation(this.program, 'a_color');
    //获取全局变量位置
    this.resolutionLocation=gl.getUniformLocation(this.program, 'u_resolution');
    this.pointSizeLocation=gl.getUniformLocation(this.program, 'u_pointSize'); }
    //创建着色器
    createShader(type, source){const gl=this.gl;
    const shader=gl.createShader(type);
    gl.shaderSource(shader, source);
    gl.compileShader(shader);
    if(!gl.getShaderParameter(shader, gl.COMPILE_STATUS)){throw new Error('着色器编译错
误: '+gl.getShaderInfoLog(shader)); }
    return shader; }
    //初始化缓冲区
    initBuffers(){const gl=this.gl;
    //创建位置缓冲区
    this.positionBuffer=gl.createBuffer();
    //创建颜色缓冲区
    this.colorBuffer=gl.createBuffer(); }
    //渲染波形数据
    renderWaveform(waveformData, color){const gl=this.gl;
    //使用着色器程序
    gl.useProgram(this.program);
    //设置视口
    gl.viewport(0, 0, gl.canvas.width, gl.canvas.height);
    //设置分辨率
    gl.uniform2f(this.resolutionLocation, gl.canvas.width, gl.canvas.height);
    //设置点大小
    gl.uniform1f(this.pointSizeLocation, 1.0);
    //准备顶点数据-修改扫描方向: 从右向左
    const positions=new Float32Array(waveformData.flatMap((point, i)=>[waveformData.length
-1-i, point]));
    //准备颜色数据
    const colors=new Float32Array(waveformData.length*4).fill(0);
    for(let i=0;i<waveformData.length;i++){const baseIndex=i*4;
    colors[baseIndex]=color[0];//R
    colors[baseIndex+1]=color[1];//G
    colors[baseIndex+2]=color[2];//B
    colors[baseIndex+3]=color[3];//A }
    //绑定并填充位置缓冲区
    gl.bindBuffer(gl.ARRAY_BUFFER, this.positionBuffer);
    gl.bufferData(gl.ARRAY_BUFFER, positions, gl.STATIC_DRAW);

```

```

gl.enableVertexAttribArray(this.positionLocation);
gl.vertexAttribPointer(this.positionLocation, 2, gl.FLOAT, false, 0, 0);
//绑定并填充颜色缓冲区
gl.bindBuffer(gl.ARRAY_BUFFER, this.colorBuffer);
gl.bufferData(gl.ARRAY_BUFFER, colors, gl.STATIC_DRAW);
gl.enableVertexAttribArray(this.colorLocation);
gl.vertexAttribPointer(this.colorLocation, 4, gl.FLOAT, false, 0, 0);
//绘制
gl.drawArrays(gl.LINE_STRIP, 0, waveformData.length); }
//清除画布
clear(color=[0, 0, 0, 1]){const gl=this.gl;
gl.clearColor(...color);
gl.clear(gl.COLOR_BUFFER_BIT); } }

```

CRTShell.jsCRT外壳页面

```

import*asTHREEfrom'three';
import{CONFIG}from'../configLoader';
import{CylinderConnection}from'./CylinderConnection.js';
import{SuperellipseTransition}from'./SuperellipseTransition.js';
import{Cylinder2ExplodeEffect}from'./Cylinder2ExplodeEffect.js';
import{RotationCurveExplodeEffect}from'./RotationCurveExplodeEffect.js';
import{SuperellipseExplodeEffect}from'./SuperellipseExplodeEffect.js';
import{unifiedComponentMaterial}from'../materials/UnifiedComponentMaterial.js';
/**
 *CRT正方形透明外壳组件
 *用于包裹整个CRT模型的透明保护壳 */
exportclassCRTShell{constructor(){this.shellGroup=newTHREE.Group();
this.shellGroup.name='CRTShell';
//创建两个圆柱体的材质
this.cylinder1Material=null;
this.cylinder2Material=null;
//旋转曲线连接
this.cylinderConnection=null;
//超椭圆形状渐变
this.superellipseTransition=null;
//cylinder2爆炸效果
this.cylinder2ExplodeEffect=null;
//旋转曲线连接爆炸效果
this.rotationCurveExplodeEffect=null;
//超椭圆形状渐变爆炸效果
this.superellipseExplodeEffect=null;
this.createShell(); }
/**
 *创建正方形透明外壳（删除靠近荧光屏的右侧面） */

```

```

createShell(){//立方体外壳材质已删除，不再需要
//this.shellMaterial=newTHREE.MeshPhongMaterial({//transparent: true,
//opacity: CONFIG.shell.opacity,
//side: THREE.DoubleSide,
//depthWrite: false
//});
//立方体外壳已被删除-只保留圆柱体和过渡效果
//constshellGeometry=this.createOpenBoxGeometry(
//CONFIG.shell.size.width,
//CONFIG.shell.size.height,
//CONFIG.shell.size.depth
//);
//constshell=newTHREE.Mesh(shellGeometry, this.shellMaterial);
//
////设置位置居中
//shell.position.set(4.5, 0, 0);//稍微向前偏移以居中包裹CRT
//
//this.shellGroup.add(shell);
//创建两个独立的圆柱体
this.createCylinder1();
this.createCylinder2();
//创建旋转曲线连接
this.createRotationCurveConnection();
//创建超椭圆形状渐变
this.createSuperellipseTransition();
//设置初始可见性
this.shellGroup.visible=CONFIG.shell.visible; }
/**
*创建开放式盒子几何体（去掉右侧面，即X正方向的面）
*@param{number}width-宽度
*@param{number}height-高度
*@param{number}depth-深度
*@returns{THREE.BufferGeometry}几何体 */
createOpenBoxGeometry(width, height, depth){constgeometry=newTHREE.BufferGeome
try();
constw=width/2;
consth=height/2;
constd=depth/2;
//定义顶点位置-立方体的8个顶点
constvertices=[
//左侧面(X=-w)
-w, -h, -d, -w, -h, d, -w, h, d,
-w, -h, -d, -w, h, d, -w, h, -d,
//右侧面被删除-不添加这个面

```

```

//顶面(Y=h)
-w, h, -d, -w, h, d, w, h, d,
-w, h, -d, w, h, d, w, h, -d,
//底面(Y=-h)
-w, -h, -d, w, -h, -d, w, -h, d,
-w, -h, -d, w, -h, d, -w, -h, d,
//前面(Z=d)
-w, -h, d, w, -h, d, w, h, d,
-w, -h, d, w, h, d, -w, h, d,
//后面(Z=-d)
-w, -h, -d, -w, h, -d, w, h, -d,
-w, -h, -d, w, h, -d, w, -h, -d
];
//定义法向量
const normals=[
//左侧面
-1, 0, 0, -1, 0, 0, -1, 0, 0,
-1, 0, 0, -1, 0, 0, -1, 0, 0,
//顶面
0, 1, 0, 0, 1, 0, 0, 1, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0,
//底面
0, -1, 0, 0, -1, 0, 0, -1, 0,
0, -1, 0, 0, -1, 0, 0, -1, 0,
//前面
0, 0, 1, 0, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 1, 0, 0, 1,
//后面
0, 0, -1, 0, 0, -1, 0, 0, -1,
0, 0, -1, 0, 0, -1, 0, 0, -1
];
//定义UV坐标
const uvs=[
//左侧面
0, 0, 1, 0, 1, 1,
0, 0, 1, 1, 0, 1,
//顶面
0, 0, 1, 0, 1, 1,
0, 0, 1, 1, 0, 1,
//底面
0, 0, 1, 0, 1, 1,
0, 0, 1, 1, 0, 1,
//前面
0, 0, 1, 0, 1, 1,

```

```

0, 0, 1, 1, 0, 1,
//后面
0, 0, 1, 0, 1, 1,
0, 0, 1, 1, 0, 1
];
geometry.setAttribute('position', newTHREE.Float32BufferAttribute(vertices, 3));
geometry.setAttribute('normal', newTHREE.Float32BufferAttribute(normals, 3));
geometry.setAttribute('uv', newTHREE.Float32BufferAttribute(uvs, 2));
return geometry; }
/**
 *创建开放式圆柱体几何体（沿X轴方向，去掉右端圆形面）
 *@param{number}radius-圆柱体半径
 *@param{number}height-圆柱体长度（沿X轴）
 *@param{number}radialSegments-径向分段数
 *@returns{THREE.BufferGeometry}几何体 */
createOpenCylinderGeometry(radius, height, radialSegments=32){//使用Three.js标准圆柱
体几何体作为基础，确保法向量和光照计算正确
//注意：openEnded设为false，这样两端都有封闭面，然后我们手动移除右端面
const baseGeometry=newTHREE.CylinderGeometry(radius, radius, height, radialSegment
s, 1, false);
//旋转90度使圆柱体沿X轴方向
baseGeometry.rotateZ(Math.PI/2);
//获取几何体属性
const positions=baseGeometry.getAttribute('position');
const normals=baseGeometry.getAttribute('normal');
const uvs=baseGeometry.getAttribute('uv');
const indices=baseGeometry.getIndex();
//创建新的几何体，只保留侧面和左端面
const newGeometry=newTHREE.BufferGeometry();
//复制侧面顶点（Three.js圆柱体的侧面已经有正确的法向量）
const newPositions=[];
const newNormals=[];
const newUvs=[];
const newIndex=[];
const posArray=positions.array;
const normArray=normals.array;
const uvArray=uvs.array;
const indexArray=indices.array;
//复制所有顶点和法向量（Three.js已经计算好了）
for(let i=0;i<posArray.length;i+=3){newPositions.push(posArray[i], posArray[i+1], posArr
ay[i+2]);
newNormals.push(normArray[i], normArray[i+1], normArray[i+2]); }
for(let i=0;i<uvArray.length;i+=2){newUvs.push(uvArray[i], uvArray[i+1]); }
//过滤索引，只保留侧面和左端面，去掉右端面

```

```

const halfHeight = height / 2;
for (let i = 0; i < indexArray.length; i += 3) {
    const i1 = indexArray[i] * 3;
    const i2 = indexArray[i + 1] * 3;
    const i3 = indexArray[i + 2] * 3;
    // 检查三角形是否在右端面 (X > halfHeight - 0.01)
    const x1 = posArray[i1];
    const x2 = posArray[i2];
    const x3 = posArray[i3];
    // 如果三角形不在右端面, 则保留
    if (!(x1 > halfHeight - 0.01 && x2 > halfHeight - 0.01 && x3 > halfHeight - 0.01)) {
        newIndices.push(indexArray[i], indexArray[i + 1], indexArray[i + 2]);
    }
    // 设置几何体属性
    newGeometry.setAttribute('position', newTHREE.Float32BufferAttribute(newPositions, 3));

    newGeometry.setAttribute('normal', newTHREE.Float32BufferAttribute(newNormals, 3));

    newGeometry.setAttribute('uv', newTHREE.Float32BufferAttribute(newUvs, 2));
    newGeometry.setIndex(newIndices);
    // 重新计算法向量以确保光照正确
    newGeometry.computeVertexNormals();
    return newGeometry;
}

/**
 * 创建第一个圆柱体组件 */
createCylinder1() {
    const config = CONFIG.shell.cylinder1;
    // 使用统一组件材质管理器获取标准材质, 确保颜色一致性
    this.cylinder1Material = unifiedComponentMaterial.getMaterial('standard');
    // 创建圆柱体几何体 (使用独立的半径参数)
    const cylinder1Geometry = newTHREE.CylinderGeometry(config.radius, config.radius, config.height, 32);
    const cylinder1 = newTHREE.Mesh(cylinder1Geometry, this.cylinder1Material);
    // 设置圆柱体位置和旋转
    cylinder1.position.set(config.position.x, config.position.y, config.position.z);
    cylinder1.rotation.set(config.rotation.x, config.rotation.y, config.rotation.z);
    // 设置可见性
    cylinder1.visible = config.visible;
    // 添加到组中
    this.shellGroup.add(cylinder1);
    // 保存圆柱体引用以便后续操作
    this.cylinder1 = cylinder1;
}

/**
 * 创建第二个圆柱体组件 (去掉右侧圆形面) */
createCylinder2() {
    const config = CONFIG.shell.cylinder2;
    // 使用统一组件材质管理器获取标准材质, 确保颜色一致性
    this.cylinder2Material = unifiedComponentMaterial.getMaterial('standard');

```

```

//创建开放式圆柱体几何体（去掉右侧圆形面）
constcylinder2Geometry=this.createOpenCylinderGeometry(config.radius, config.height, 3
2);
constcylinder2=newTHREE.Mesh(cylinder2Geometry, this.cylinder2Material);
//设置圆柱体位置和旋转
cylinder2.position.set(config.position.x, config.position.y, config.position.z);
cylinder2.rotation.set(config.rotation.x, config.rotation.y, config.rotation.z);
//设置可见性
cylinder2.visible=config.visible;
//添加到组中
this.shellGroup.add(cylinder2);
//保存圆柱体引用以便后续操作
this.cylinder2=cylinder2;
//创建cylinder2的爆炸效果
this.createCylinder2ExplodeEffect(); }
/**
*创建旋转曲线连接 */
createRotationCurveConnection(){constconfig=CONFIG.shell.rotationCurveConnection;
if(!config||!config.visible){return; }
//创建圆柱体连接
this.cylinderConnection=newCylinderConnection(
CONFIG.shell.cylinder1,
CONFIG.shell.cylinder2,
config
);
//添加到组中
this.shellGroup.add(this.cylinderConnection.getConnection());
//创建旋转曲线连接爆炸效果
this.createConnectionExplodeEffect(); }
/**
*创建超椭圆形状渐变 */
createSuperellipseTransition(){constconfig=CONFIG.shell.superellipseTransition;
if(!config||!config.visible){return; }
//准备起始配置（第一个圆柱体）
conststartConfig={position: CONFIG.shell.cylinder1.position,
radius: CONFIG.shell.cylinder1.radius };
//准备结束配置（适配荧光屏尺寸）
constendConfig={position: {x: 4.5, y: 0, z: 0}, //立方体中心位置
size: {width: CONFIG.components.screen.width, //荧光屏宽度: 4
height: CONFIG.components.screen.height, //荧光屏高度: 3
depth: CONFIG.shell.size.depth//保持原深度: 4 } };
//创建形状渐变
this.superellipseTransition=newSuperellipseTransition(
startConfig,

```

```

endConfig,
config
);
//添加到组中
this.shellGroup.add(this.superellipseTransition.getTransition());
//创建超椭圆形状渐变爆炸效果
this.createSuperellipseExplodeEffect(); }
/**
 *创建旋转曲线连接爆炸效果 */
createConnectionExplodeEffect(){if(!this.cylinderConnection){console.warn('CylinderConne
ctionnotfound, cannotcreateexplodeeffect');
return; }
//清理旧的爆炸效果实例
if(this.rotationCurveExplodeEffect){this.rotationCurveExplodeEffect.dispose();
this.rotationCurveExplodeEffect=null; }
//获取连接网格（第一个连接网格）
constconnectionMeshes=this.cylinderConnection.connectionMeshes;
if(!connectionMeshes||connectionMeshes.length===0){console.warn('Noconnectionmeshesfo
und');
return; }
//创建爆炸效果实例
this.rotationCurveExplodeEffect=newRotationCurveExplodeEffect(
connectionMeshes[0], //使用第一个连接网格
this.shellGroup
); }
/**
 *创建cylinder2爆炸效果 */
createCylinder2ExplodeEffect(){if(!this.cylinder2){console.warn('Cylinder2notfound, canno
tcreateexplodeeffect');
return; }
//清理旧的爆炸效果实例
if(this.cylinder2ExplodeEffect){this.cylinder2ExplodeEffect.dispose();
this.cylinder2ExplodeEffect=null; }
//创建爆炸效果实例
this.cylinder2ExplodeEffect=newCylinder2ExplodeEffect(
this.cylinder2,
this.shellGroup
); }
/**
 *创建超椭圆形状渐变爆炸效果 */
createSuperellipseExplodeEffect(){if(!this.superellipseTransition){console.warn('Superellips
eTransitionnotfound, cannotcreateexplodeeffect');
return; }
//清理旧的爆炸效果实例

```

```

if(this.superellipseExplodeEffect){this.superellipseExplodeEffect.dispose();
this.superellipseExplodeEffect=null; }
//获取超椭圆的网格（第一个网格）
consttransitionMeshes=this.superellipseTransition.meshes;
if(!transitionMeshes||transitionMeshes.length===0){console.warn('Nosuperellipsemeshesfound');
return; }
//创建爆炸效果实例
this.superellipseExplodeEffect=newSuperellipseExplodeEffect(
transitionMeshes[0], //使用第一个网格
this.shellGroup
); }
/**
*切换cylinder2爆炸效果
*@param{boolean}explode-是否爆炸，默认为切换状态
*@returns{boolean}当前爆炸状态 */
toggleCylinder2Explode(explode){if(!this.cylinder2ExplodeEffect){console.warn('Cylinder2
explodeeffectnotinitialized');
returnfalse; }
returnthis.cylinder2ExplodeEffect.toggle(explode); }
/**
*获取cylinder2爆炸状态
*@returns{boolean}是否处于爆炸状态 */
isCylinder2Exploded(){returnthis.cylinder2ExplodeEffect?this.cylinder2ExplodeEffect.isExploded(): false; }
/**
*切换旋转曲线连接爆炸效果
*@param{boolean}explode-是否爆炸，默认为切换状态
*@returns{boolean}当前爆炸状态 */
toggleConnectionExplode(explode){if(!this.rotationCurveExplodeEffect){console.warn('RotationCurveexplodeeffectnotinitialized');
returnfalse; }
returnthis.rotationCurveExplodeEffect.toggle(explode); }
/**
*获取旋转曲线连接爆炸状态
*@returns{boolean}是否处于爆炸状态 */
isConnectionExploded(){returnthis.rotationCurveExplodeEffect?this.rotationCurveExplodeEffect.isExploded(): false; }
/**
*切换超椭圆形状渐变爆炸效果
*@param{boolean}explode-是否爆炸，默认为切换状态
*@returns{boolean}当前爆炸状态 */
toggleSuperellipseExplode(explode){if(!this.superellipseExplodeEffect){console.warn('Superellipseexplodeeffectnotinitialized');

```

```

return false; }
return this.superellipseExplodeEffect.toggle(explode); }
/**
 * 获取超椭圆形状渐变爆炸状态
 * @returns {boolean} 是否处于爆炸状态 */
isSuperellipseExploded() { return this.superellipseExplodeEffect.isExploded(); false; }
/**
 * 获取外壳组 */
getShell() { return this.shellGroup; }
/**
 * 设置外壳可见性 */
setVisible(visible) { this.shellGroup.visible = visible; }
/**
 * 设置外壳透明度 */
setOpacity(opacity) { // 立方体外壳已删除，此方法不再有效
// this.shellMaterial.opacity = Math.max(0, Math.min(1, opacity)); }
/**
 * 设置外壳位置 */
setPosition(x, y, z) { this.shellGroup.position.set(x, y, z); }
/**
 * 设置第一个圆柱体位置 */
setCylinder1Position(x, y, z) { if (this.cylinder1) { this.cylinder1.position.set(x, y, z); } }
/**
 * 设置第二个圆柱体位置 */
setCylinder2Position(x, y, z) { if (this.cylinder2) { this.cylinder2.position.set(x, y, z); } }
/**
 * 获取外壳当前位置 */
getPosition() { return this.shellGroup.position.clone(); }
/**
 * 更新动画-外壳设置为静态，无动画效果 */
update(deltaTime) { // 外壳保持静态，不进行任何动画更新
// 如果需要动画效果，可以在这里添加 }
/**
 * 更新配置-实现响应式配置更新 */
updateConfig() { // 立方体外壳材质已删除，不再需要更新
// if (this.shellMaterial) { this.shellMaterial.opacity = CONFIG.shell.opacity;
// }
// 更新第一个圆柱体配置
if (this.cylinder1Material && CONFIG.shell.cylinder1) { this.cylinder1Material.color.setHex(parseInt(CONFIG.shell.cylinder1.color));
this.cylinder1Material.opacity = CONFIG.shell.cylinder1.opacity; }
if (this.cylinder1 && CONFIG.shell.cylinder1) { this.cylinder1.visible = CONFIG.shell.cylinder1.visible;

```

```

this.cylinder1.position.set(
  CONFIG.shell.cylinder1.position.x,
  CONFIG.shell.cylinder1.position.y,
  CONFIG.shell.cylinder1.position.z
);
this.cylinder1.rotation.set(
  CONFIG.shell.cylinder1.rotation.x,
  CONFIG.shell.cylinder1.rotation.y,
  CONFIG.shell.cylinder1.rotation.z
); }
//更新第二个圆柱体配置
if(this.cylinder2Material&&CONFIG.shell.cylinder2){this.cylinder2Material.color.setHex(parseInt(CONFIG.shell.cylinder2.color));
  this.cylinder2Material.opacity=CONFIG.shell.cylinder2.opacity; }
if(this.cylinder2&&CONFIG.shell.cylinder2){this.cylinder2.visible=CONFIG.shell.cylinder2.visible;
  this.cylinder2.position.set(
    CONFIG.shell.cylinder2.position.x,
    CONFIG.shell.cylinder2.position.y,
    CONFIG.shell.cylinder2.position.z
  );
  this.cylinder2.rotation.set(
    CONFIG.shell.cylinder2.rotation.x,
    CONFIG.shell.cylinder2.rotation.y,
    CONFIG.shell.cylinder2.rotation.z
  ); }
//更新旋转曲线连接配置
if(this.cylinderConnection&&CONFIG.shell.rotationCurveConnection){this.cylinderConnection.updateConfig(
  CONFIG.shell.cylinder1,
  CONFIG.shell.cylinder2,
  CONFIG.shell.rotationCurveConnection
); }elseif(!this.cylinderConnection&&CONFIG.shell.rotationCurveConnection?.visible){//
如果连接不存在但配置中要求显示，则创建连接
  //注意：这里不应该重复创建，因为可能导致重复模型
  console.warn('CylinderConnectionnotfound, butconfigrequiresittobevisible'); }
//更新超椭圆形状渐变配置
if(this.superellipseTransition&&CONFIG.shell.superellipseTransition){conststartConfig={position: CONFIG.shell.cylinder1.position,
  radius: CONFIG.shell.cylinder1.radius };
constendConfig={position: {x: 4.5, y: 0, z: 0},
  size: {width: CONFIG.components.screen.width, //荧光屏宽度: 4
  height: CONFIG.components.screen.height, //荧光屏高度: 3
  depth: CONFIG.shell.size.depth//保持原深度: 4 } };

```

```

this.superellipseTransition.updateConfig(
startConfig,
endConfig,
CONFIG.shell.superellipseTransition
); }elseif(!this.superellipseTransition&&CONFIG.shell.superellipseTransition?.visible){//如果
形状渐变不存在但配置中要求显示，则创建渐变
//注意：这里不应该重复创建，因为可能导致重复模型
console.warn('SuperellipseTransitionnotfound, butconfigrequiresittobevisible'); }
//更新可见性
this.shellGroup.visible=CONFIG.shell.visible; }
/**
*设置第一个圆柱体旋转 */
setCylinder1Rotation(x, y, z){if(this.cylinder1){this.cylinder1.rotation.set(x, y, z); } }
/**
*设置第二个圆柱体旋转 */
setCylinder2Rotation(x, y, z){if(this.cylinder2){this.cylinder2.rotation.set(x, y, z); } }
/**
*获取第一个圆柱体当前旋转 */
getCylinder1Rotation(){returnthis.cylinder1?this.cylinder1.rotation.clone(): newTHREE.Euler(); }
/**
*获取第二个圆柱体当前旋转 */
getCylinder2Rotation(){returnthis.cylinder2?this.cylinder2.rotation.clone(): newTHREE.Euler(); }
/**
*设置第一个圆柱体可见性 */
setCylinder1Visible(visible){if(this.cylinder1){this.cylinder1.visible=visible; } }
/**
*设置第二个圆柱体可见性 */
setCylinder2Visible(visible){if(this.cylinder2){this.cylinder2.visible=visible; } }
/**
*设置第一个圆柱体颜色 */
setCylinder1Color(color){if(this.cylinder1Material){this.cylinder1Material.color.setHex(color); } }
/**
*设置第二个圆柱体颜色 */
setCylinder2Color(color){if(this.cylinder2Material){this.cylinder2Material.color.setHex(color); } }
/**
*设置第一个圆柱体透明度 */
setCylinder1Opacity(opacity){if(this.cylinder1Material){this.cylinder1Material.opacity=Math.max(0, Math.min(1, opacity)); } }
/**
*设置第二个圆柱体透明度 */

```

```

        setCylinder2Opacity(opacity){if(this.cylinder2Material){this.cylinder2Material.opacity=Math.max(0, Math.min(1, opacity)); } }
    /**
     *设置旋转曲线连接的位置偏移
     *@param{number}x-X轴偏移
     *@param{number}y-Y轴偏移
     *@param{number}z-Z轴偏移 */
    setConnectionPositionOffset(x, y, z){if(this.cylinderConnection){this.cylinderConnection.setPositionOffset(x, y, z); } }
    /**
     *设置旋转曲线连接的旋转偏移
     *@param{number}x-X轴旋转偏移（弧度）
     *@param{number}y-Y轴旋转偏移（弧度）
     *@param{number}z-Z轴旋转偏移（弧度） */
    setConnectionRotationOffset(x, y, z){if(this.cylinderConnection){this.cylinderConnection.setRotationOffset(x, y, z); } }
    /**
     *直接设置旋转曲线连接的位置
     *@param{number}x-X坐标
     *@param{number}y-Y坐标
     *@param{number}z-Z坐标 */
    setConnectionPosition(x, y, z){if(this.cylinderConnection){this.cylinderConnection.setPosition(x, y, z); } }
    /**
     *直接设置旋转曲线连接的旋转
     *@param{number}x-X轴旋转（弧度）
     *@param{number}y-Y轴旋转（弧度）
     *@param{number}z-Z轴旋转（弧度） */
    setConnectionRotation(x, y, z){if(this.cylinderConnection){this.cylinderConnection.setRotation(x, y, z); } }
    /**
     *获取旋转曲线连接的当前位置 */
    getConnectionPosition(){returnthis.cylinderConnection?this.cylinderConnection.getPosition(): newTHREE.Vector3(); }
    /**
     *获取旋转曲线连接的当前旋转 */
    getConnectionRotation(){returnthis.cylinderConnection?this.cylinderConnection.getRotation(): newTHREE.Euler(); }
    /**
     *获取旋转曲线连接的位置偏移 */
    getConnectionPositionOffset(){returnthis.cylinderConnection?this.cylinderConnection.getPositionOffset(): {x: 0, y: 0, z: 0}; }
    /**
     *获取旋转曲线连接的旋转偏移 */

```

```

    getConnectionRotationOffset(){return this.cylinderConnection?this.cylinderConnection.getRotationOffset(): {x: 0, y: 0, z: 0}; }
    /**
     *设置旋转曲线连接可见性 */
    setConnectionVisible(visible){if(this.cylinderConnection){this.cylinderConnection.setVisible(visible); } }
    /**
     *设置旋转曲线连接颜色 */
    setConnectionColor(color){if(this.cylinderConnection){this.cylinderConnection.setColor(color); } }
    /**
     *设置旋转曲线连接透明度 */
    setConnectionOpacity(opacity){if(this.cylinderConnection){this.cylinderConnection.setOpacity(opacity); } }
    /**
     *设置超椭圆形状渐变的位置偏移
     *@param{number}x-X轴偏移
     *@param{number}y-Y轴偏移
     *@param{number}z-Z轴偏移 */
    setTransitionPositionOffset(x, y, z){if(this.superellipseTransition){this.superellipseTransition.setPositionOffset(x, y, z); } }
    /**
     *设置超椭圆形状渐变的旋转偏移
     *@param{number}x-X轴旋转偏移（弧度）
     *@param{number}y-Y轴旋转偏移（弧度）
     *@param{number}z-Z轴旋转偏移（弧度） */
    setTransitionRotationOffset(x, y, z){if(this.superellipseTransition){this.superellipseTransition.setRotationOffset(x, y, z); } }
    /**
     *直接设置超椭圆形状渐变的位置
     *@param{number}x-X坐标
     *@param{number}y-Y坐标
     *@param{number}z-Z坐标 */
    setTransitionPosition(x, y, z){if(this.superellipseTransition){this.superellipseTransition.setPosition(x, y, z); } }
    /**
     *直接设置超椭圆形状渐变的旋转
     *@param{number}x-X轴旋转（弧度）
     *@param{number}y-Y轴旋转（弧度）
     *@param{number}z-Z轴旋转（弧度） */
    setTransitionRotation(x, y, z){if(this.superellipseTransition){this.superellipseTransition.setRotation(x, y, z); } }
    /**
     *获取超椭圆形状渐变的当前位置 */

```

```

    getTransitionPosition(){returnthis.superellipseTransition?this.superellipseTransition.getPositi
on(): newTHREE.Vector3(); }
    /**
    *获取超椭圆形状渐变的当前旋转 */
    getTransitionRotation(){returnthis.superellipseTransition?this.superellipseTransition.getRotat
ion(): newTHREE.Euler(); }
    /**
    *获取超椭圆形状渐变的位置偏移 */
    getTransitionPositionOffset(){returnthis.superellipseTransition?this.superellipseTransition.ge
tPositionOffset(): {x: 0, y: 0, z: 0}; }
    /**
    *获取超椭圆形状渐变的旋转偏移 */
    getTransitionRotationOffset(){returnthis.superellipseTransition?this.superellipseTransition.ge
tRotationOffset(): {x: 0, y: 0, z: 0}; }
    /**
    *设置超椭圆形状渐变可见性 */
    setTransitionVisible(visible){if(this.superellipseTransition){this.superellipseTransition.setVis
ible(visible); } }
    /**
    *设置超椭圆形状渐变颜色 */
    setTransitionColor(color){if(this.superellipseTransition){this.superellipseTransition.setColor
(color); } }
    /**
    *设置超椭圆形状渐变透明度 */
    setTransitionOpacity(opacity){if(this.superellipseTransition){this.superellipseTransition.setO
pacity(opacity); } }
    /**
    *动态更新超椭圆指数（用于动画效果）
    *@param{number}startExponent-起始指数（圆形： 2.0）
    *@param{number}endExponent-结束指数（方形： 8.0+） */
    updateTransitionExponents(startExponent, endExponent){if(this.superellipseTransition){thi
s.superellipseTransition.updateExponents(startExponent, endExponent); } }
    /**
    *销毁资源 */
    dispose(){//立方体外壳材质已删除，不需要清理
    //if(this.shellMaterial){this.shellMaterial.dispose();
    //}
    if(this.cylinder1Material){this.cylinder1Material.dispose(); }
    if(this.cylinder2Material){this.cylinder2Material.dispose(); }
    //清理旋转曲线连接
    if(this.cylinderConnection){this.cylinderConnection.dispose();
    this.cylinderConnection=null; }
    //清理超椭圆形状渐变
    if(this.superellipseTransition){this.superellipseTransition.dispose();

```

```

this.superellipseTransition=null; }
//清理cylinder2爆炸效果
if(this.cylinder2ExplodeEffect){this.cylinder2ExplodeEffect.dispose();
this.cylinder2ExplodeEffect=null; }
//清理旋转曲线连接爆炸效果
if(this.rotationCurveExplodeEffect){this.rotationCurveExplodeEffect.dispose();
this.rotationCurveExplodeEffect=null; }
//清理超椭圆形状渐变爆炸效果
if(this.superellipseExplodeEffect){this.superellipseExplodeEffect.dispose();
this.superellipseExplodeEffect=null; }
this.shellGroup.traverse((child)=>{if(child.geometry)child.geometry.dispose();
if(child.material)child.material.dispose(); });
//清理圆柱体引用
this.cylinder1=null;
this.cylinder2=null; } }

```

Screen.js 荧光屏效果和波形显示

```

import*asTHREEfrom'three';
import{CONFIG}from'../configLoader';
/**
 *荧光屏类
 *负责处理荧光屏效果和波形显示
 *集成了ThreeWaveformRenderer的波形渲染功能 */
exportclassScreen{/**
 *构造函数
 *@param{THREE.Scene} scene-Three.js场景
 *@param{THREE.Mesh} screenMesh-荧光屏网格 */
constructor(scene, screenMesh){this.scene=scene;
this.screenMesh=screenMesh;
//创建荧光点集合
this.glowPoints=[];
this.glowMeshes=[];
this.maxGlowPoints=CONFIG.screenEffects.maxGlowPoints;
//为了兼容DemoAnimation的focusOnComponent方法，添加position属性
this.position=this.screenMesh.position;
//移除了静态波形相关属性，保留电子束轨迹波形
//初始化荧光材质
this.initGlowMaterial();
//创建荧光屏网格
this.createScreenGrid();
//移除了静态波形初始化 }
/**
 *初始化荧光材质 */
initGlowMaterial(){//更新荧光屏材质

```

```

this.screenMesh.material.emissive.set(CONFIG.screen.color);
this.screenMesh.material.emissiveIntensity=CONFIG.screen.intensity;
//创建荧光点材质
this.glowMaterial=newTHREE.MeshBasicMaterial({color: CONFIG.beam.color, //使用电
子束的颜色（绿色）
transparent: true,
opacity: 0.8 });
//移除了静态波形材质 }
/**
*创建荧光屏网格 */
createScreenGrid(){//获取荧光屏的尺寸和位置
constscreenWidth=CONFIG.components.screen.width;
constscreenHeight=CONFIG.components.screen.height;
constscreenPosition=CONFIG.components.screen.position;
//创建网格材质
constgridMaterial=newTHREE.LineBasicMaterial({color: CONFIG.screen.gridColor,
transparent: true,
opacity: CONFIG.screen.gridOpacity });
//创建网格几何体
constgridGeometry=newTHREE.BufferGeometry();
constgridPoints=[];
//计算网格线间距
constgridSpacing=CONFIG.screen.gridSpacing;
consthalfWidth=screenWidth/2;
consthalfHeight=screenHeight/2;
//添加垂直线
for(letx=-halfWidth;x<=halfWidth;x+=gridSpacing){gridPoints.push(x, -halfHeight, 0);
gridPoints.push(x, halfHeight, 0); }
//添加水平线
for(lety=-halfHeight;y<=halfHeight;y+=gridSpacing){gridPoints.push(-halfWidth, y, 0);
gridPoints.push(halfWidth, y, 0); }
//设置几何体属性
gridGeometry.setAttribute('position', newTHREE.Float32BufferAttribute(gridPoints, 3));
//创建网格线
this.screenGrid=newTHREE.LineSegments(gridGeometry, gridMaterial);
//设置网格位置（稍微偏移，避免z-fighting）
this.screenGrid.position.copy(screenPosition);
this.screenGrid.position.x+=0.01;//在荧光屏前面一点点
this.screenGrid.rotation.y=-Math.PI/2;//与荧光屏相同的旋转
//添加到场景
this.scene.add(this.screenGrid); }
//移除了initWaveform方法
//移除了createWaveform方法
//移除了createSingleWaveform方法

```

```

/**
 *根据波形类型计算电压值
 *@param{string} waveType-波形类型
 *@param{number} phaseVal-相位值(弧度)
 *@param{number} amplitude-振幅
 *@returns{number} 电压值 */
calculateVoltage(waveType, phaseVal, amplitude){switch(waveType){case'sine':
returnamplitude*Math.sin(phaseVal);
case'square':
returnamplitude*(Math.sin(phaseVal)>=0?1: -1);
case'triangle':
returnamplitude*(2*Math.abs((phaseVal%(2*Math.PI))/(2*Math.PI)-0.5)-1);
case'sawtooth':
returnamplitude*(((phaseVal%(2*Math.PI))/(2*Math.PI))*2-1);
default:
return0; } }
/**
 *添加荧光点
 *@param{THREE.Vector3} position-电子束击中荧光屏的位置 */
addGlowPoint(position){//创建一个球体代表荧光点
constglowGeometry=newTHREE.SphereGeometry(CONFIG.screenEffects.glowPointSize,
6, 6);
constglowMesh=newTHREE.Mesh(glowGeometry, this.glowMaterial.clone());
//设置位置（稍微偏移，避免z-fighting）
glowMesh.position.copy(position);
constscreenOffset=CONFIG.electronBeam.screenOffset;
glowMesh.position.x=CONFIG.components.screen.position.x-screenOffset;//确保在荧光屏
前面一点点
//添加到场景
this.scene.add(glowMesh);
//记录荧光点信息
this.glowPoints.push({mesh: glowMesh,
createdAt: Date.now(),
initialOpacity: 0.8 });
//如果荧光点过多，移除最早的点
if(this.glowPoints.length>this.maxGlowPoints){constoldestPoint=this.glowPoints.shift();
this.scene.remove(oldestPoint.mesh);
oldestPoint.mesh.geometry.dispose();
oldestPoint.mesh.material.dispose(); } }
/**
 *更新荧光点效果和波形动画 */
update(){constnow=Date.now();
constpersistence=CONFIG.screen.persistence;
//更新荧光屏材质

```

```

this.screenMesh.material.emissive.set(CONFIG.screen.color);
this.screenMesh.material.emissiveIntensity=CONFIG.screen.intensity;
//更新每个荧光点的透明度（模拟余辉效果）
this.glowPoints.forEach((point, index)=>{constage=(now-point.createdAt)/1000;//年龄（秒）

constfadeRate=1-persistence;//淡出速率
constopacity=point.initialOpacity*Math.pow(persistence, age*CONFIG.screenEffects.fadeRate);
//更新透明度
point.mesh.material.opacity=opacity;
//如果完全透明，移除这个点
if(opacity<CONFIG.screenEffects.minOpacity){this.scene.remove(point.mesh);
point.mesh.geometry.dispose();
point.mesh.material.dispose();
this.glowPoints.splice(index, 1); } });
//移除了静态波形动画逻辑 }
/**
*清除所有荧光点 */
clearAllGlowPoints(){this.glowPoints.forEach(point=>{this.scene.remove(point.mesh);
point.mesh.geometry.dispose();
point.mesh.material.dispose(); });
this.glowPoints=[]; }
/**
*更新材质颜色 */
updateMaterial(){//更新荧光屏材质
this.screenMesh.material.emissive.set(CONFIG.screen.color);
this.screenMesh.material.emissiveIntensity=CONFIG.screen.intensity;
//更新所有荧光点的颜色
this.glowPoints.forEach(point=>{point.mesh.material.color.set(CONFIG.screen.color); }); }
//=====移除了所有静态波形控制方法=====
//保留电子束轨迹波形，这些方法不再需要
/**
*显示/隐藏波形-保留此方法以避免调用错误，但不执行任何操作
*@param{boolean}show-是否显示波形 */
showWaveform(show){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
*设置波形类型-保留此方法以避免调用错误，但不执行任何操作
*@param{string}type-波形类型 */
setWaveformType(type){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
*设置频率-保留此方法以避免调用错误，但不执行任何操作
*@param{number}frequency-频率值 */

```

```
setFrequency(frequency){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
 *设置振幅-保留此方法以避免调用错误，但不执行任何操作
 *@param{number}amplitude-振幅值 */
setAmplitude(amplitude){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
 *设置相位-保留此方法以避免调用错误，但不执行任何操作
 *@param{number}phase-相位值（弧度） */
setPhase(phase){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
 *开始波形动画-保留此方法以避免调用错误，但不执行任何操作 */
startWaveformAnimation(){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; }
/**
 *停止波形动画-保留此方法以避免调用错误，但不执行任何操作 */
stopWaveformAnimation(){//静态波形已移除，此方法保留为空以避免调用错误
returnthis; } }
```